Davao del Sur State College
Brgy. Matti, Digos City, Davao del Sur

Object-Oriented Programming

Version 2.0



www.dssc.edu.ph @DSSCOfficial president@dssc.edu.ph

Object-Oriented Programming

Version 2.0

A Course Pack for
BSIT, BSIS and DIT Students

Razel Ann G. Aguilar, MIT
Davao del Sur State College
Matti, Digos City, Davao del Sur

COURSE OVERVIEW

Hello dear students! Welcome to your journey into Object-Oriented Programming (OOP), an exciting step forward from the programming concepts you already know. In this course, we'll build on your background in procedural programming and guide you toward a more powerful and organized way of creating programs.

We'll begin by reviewing the basic concepts of conditions and loops, as well as data types, with an emphasis on structured data types, arrays, pointers, and functions. From there, we'll gradually introduce you to the world of object-oriented programming, where you'll learn how real-world problems are modeled using classes, objects, and methods. Along the way, you'll discover core OOP principles like abstraction, encapsulation, inheritance, and polymorphism, which are essential skills for modern software development. You'll also explore exception handling, templates, file handling, and database connectivity to make your programs more reliable and practical.

This course pack is organized into four modules designed to guide you step by step:

Module 1: Introduction to Object-oriented Programming
Module 2: Object-Oriented Programming in Java
Module 3: Object-Oriented Design Principles
Module 4: File Handling and Database Connectivity

At the end of this course pack, the learner will be able to:

1. Compare and contrast the procedural approach with the object-oriented programming approach.
2. Design, implement, test, and debug programs using OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.

To get the most out of this learning experience, we encourage you to complete each lesson in sequence, as every topic builds upon the previous one. Take your time, collaborate with your classmates, and enjoy the learning process. Remember, you're not alone in this journey. Feel free to ask your instructors, or classmates for guidance and support whenever you need it.

Good luck, have fun, and let's start coding!

PREFACE

In today's fast-changing world of technology, dear students, learning object-oriented programming (OOP) is more than just picking up new coding skills, it's about developing a new way to think, design, and solve problems. Moving from procedural programming to OOP allows you to view programs as collections of interacting objects , much like how systems work in the real world.

This course is designed to guide you step by step in building a strong foundation in object-oriented concepts, which are at the core of modern software development. This course pack was carefully developed to provide you with a clear and structured learning pathway, using Java as the primary programming language to support your growth and understanding.

Throughout this course pack, learning is progressive and practical. Each module builds upon the previous one through examples, exercises, and hands-on coding activities that reinforce key ideas. You are encouraged to actively engage with the lessons and apply what you learn through practice, as this will strengthen your problem-solving skills and deepen your appreciation for well-designed object-oriented software.

To make the most of this course pack, students are encouraged to:

- Follow the modules sequentially to build a coherent and cumulative understanding of OOP.
- Practice consistently by working on the coding examples and activities provided in each module.
- Engage with peers and instructors to discuss concepts, troubleshoot challenges, and exchange ideas.
- Review and revisit previous modules if concepts feel unclear or need reinforcement before moving on.

This course pack is more than a collection of theories, it is a learning companion for you, dear students, designed to build confidence and competence in object-oriented programming. Learning OOP may come with challenges, but with practice, collaboration, and persistence, you will see how it simplifies complex problems and inspires creative solutions. Move at your own pace as you explore the concepts, practice the code, and ask questions whenever needed. Your journey into object-oriented programming begins here.

Institute of Computing, Engineering and Technology
Computing Department
First Semester, Academic Year 2024-2025

COURSE OUTLINE

Course Name: **IT 622 – OBJECT ORIENTED PROGRAMMING**
Course Credits: **3.0**
Course Description: This course introduces object-oriented programming concepts to students with a background in the procedural paradigm. The course begins with a review of the basic concepts of conditionals and loops, and data types with an emphasis on structured data types, arrays, pointers, and functions. It then evolves on the object-oriented programming paradigm, focusing on the definition and use of classes, objects, and methods together with the fundamentals of object-oriented design. Other areas include fundamentals of OOP concepts, templates, exceptions, and file handling.
Contact Hours/ Week: Lecture (2 hours per week) Laboratory (3 hours per week)
Pre-requisite: CS 612 – Computing Programming 2
Course Outcomes: 1. CO1. Compare and contrast the procedural approach with the object-oriented programming approach
2. CO2. Design, implement, test, and debug programs using OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.

COURSE OUTLINE AND TIMEFRAME

Week	Topic and Sub-Topics	Student Learning Outcomes
1	**Course Introduction & Institutional Orientation** DSSC's Vision, Mission, and Core Values Classroom Policies, Grading System Course Platform & Tools Setup	At the end of the lesson, the students will be able to: • Recall the Vision, Mission, Goals, and Core Values of the institution. • Appreciate the relevance of the institution's VGMO. • Be oriented in the class policies, course requirements, course assessment, and grading system MAP: Institutional Alignment
	Module 1 – Introduction to	At the end of the lesson, the students will be able to: • Analyze the key characteristics, advantages, and limitations of

2	**Object-Oriented Programming** Lesson 1: Procedural vs. Object-oriented Approach **Lab Activity:** Comparative Analysis of Procedural and OOP using tables	procedural and object-oriented programming. • Demonstrate appreciation for the principles of both approaches by recognizing their appropriate use in real-world scenarios. • Construct simple code examples that demonstrate both procedural and object-oriented approaches. MAP: CO1
3-5	**Module 1 – Introduction to Object-Oriented Programming** Lesson 2: The building Blocks of Java: A Review of Fundamentals Concepts **Lab Activity:** • Hands-on Lab Activity (Instructor demo followed by student practice) • Pair programming activity	At the end of the lesson, the students will be able to: • Explain and apply the fundamental concepts of Java including tokens, control structures, and error types in simple code • Show a positive attitude toward writing clean and error-free Java code • Write and debug Java programs using proper syntax, control flow and error handling. MAP: CO1
6	**Module 2 – Object-Oriented Programming in Java** Lesson 1: Objects and Classes **Lab Activity:**	At the end of the lesson, the students will be able to: • Explain and apply the fundamental concepts of Java including tokens, control structures, and error types in simple code • Show a positive attitude toward writing clean and error-free Java code

	Hands-on Coding Activities on objects and classes	• Write and debug Java programs using proper syntax, control flow and error handling. MAP: CO1
7-8	**Module 2 – Object-Oriented Programming in Java** Lesson 2: Encapsulation **Lab Activity:** Hands-on Coding Activities implementing encapsulation in Java	At the end of the lesson, the students will be able to: • Explain the concept of encapsulation and apply access modifiers to control data visibility and access in Java classes. • Recognize the importance of encapsulation in promoting data security • Implement encapsulation in Java by defining attributes as private and providing appropriate getter and setter methods. MAP: C02
9	**Midterm Examination** Covers Modules 1 and 2 Lecture and Laboratory Exam	Evaluate students' knowledge and skills relevant to all topics discussed in Modules 1 and 2.
10	**Module 3 – Object-Oriented Programming Design Principles** Lesson 1: Abstraction **Lab Activity:** Hands-on coding activities implementing abstraction in Java	At the end of the lesson, the students will be able to: • Explain the concept of abstraction and differentiate it from encapsulation, with examples of abstract classes and interfaces in Java. • Recognize the value of abstraction in simplifying complex systems • Design and implement Java programs using abstract classes or interfaces. MAP: CO2

11-12	**Module 3 – Object-Oriented Programming Design Principles** Lesson 2: Inheritance **Lab Activity:** • Hands-on coding activities implementing inheritance in Java • Create superclasses and multiple subclasses	At the end of the lesson, the students will be able to: • Explain the concept of inheritance and analyze how subclasses acquire properties and behaviors from superclasses in Java. • Recognize the value of inheritance in reducing redundancy, and express willingness to reuse code through superclass design in programming. • Implement inheritance in Java by creating subclasses that inherit from a superclass. MAP: CO2
13-14	**Module 3 – Object-Oriented Programming Design Principles** Lesson 3: Polymorphism **Lab Activity:** • Hands-on coding activities implementing polymorphism in Java	At the end of the lesson, the students will be able to: • Explain the concept of polymorphism and distinguish between method overloading and method overriding in Java. • Analyze how polymorphism enhances flexibility and scalability in software development. • Implement polymorphism in Java programs using method overriding and overloading. MAP: CO2
	Module 4– File Handling and Database Connectivity Lesson 1: File Input/Output Streams	At the end of the lesson, the students will be able to: • Explain the purpose and types of file input/output streams in Java and differentiate between byte and character streams.

15	**Lab Activity:** • Hands-on coding activities performing file reading and writing in Java.	• Demonstrate an understanding of the importance of file handling in building real-world applications. • Write Java programs that use file input/output streams to read from and write to files using appropriate classes and exception handling. MAP: C02
16-17	**Module 4– File Handling and Database Connectivity** Lesson 2: Database Connectivity **Lab Activity:** • Hands-on coding activities using JDBC to read and display data in Java.	At the end of the lesson, the students will be able to: • Explain the concept of database connectivity and describe how Java Database Connectivity (JDBC) is used to connect Java applications with relational databases • Show appreciation for the role of database connectivity in building dynamic applications. • Develop a simple Java program that connects to a database, retrieves, and displays it using JDBC. MAP: C02
18	**Final Examination** Covers Modules 3 and 4 Lecture and Laboratory Exam	Evaluate students' knowledge and skills relevant to all topics discussed in Modules 3 and 4.
References	1. Aguilar, R. (2024). *Object-oriented programming version 2.0.* Davao del Sur State College. 2. Burd, B. (2021). *Beginning programming with java for dummies.* John Wiley & Sons, Inc. 3. Budd, T.A. (2021). *An introduction to object-oriented programming.* Addison Wesley. 4. Davao del Sur State College. (2024). *Student handbook(1st ed.).* 5. Sarcar, V. (2020). *Interactive object-oriented programming in java.* Springer Science. 6. 3G E-learning. (2023). *Object oriented programming.*	

Grading System:	Distribution of percentage for the computation of grades on subjects with laboratory is as follows: Laboratory Activities 60% Non-Laboratory Activities 40% **Total 100%** Computation of Final Grade: **Midterm Grade 50%** **Final Term Grade 50%** **Final Grade = (MTG + FTG)/2** **Passing Percentage: 50%**

Prepared by:

RAZEL ANN G. AGUILAR
Faculty

Checked by:

DOMINGO V. ORIGINES, IT.D
Department Chairperson

Approved by:

EDUARDO F. AQUINO, MS
Institute Dean

TABLE OF CONTENTS

Course Overview i

Preface ii

Course Outline iii

Table of Contents ix

MODULE 1: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING 1

Module 1 Lesson 1: Procedural vs Object-Oriented Programming 2

Module 1 Lesson 2: The Building Blocks of Java : A Review of Fundamentals Concept 6

Module Assessment 13

Module Summary 14

References 15

MODULE 2: OBJECT-ORIENTED PROGRAMMING IN JAVA 16

Module 2 Lesson 1:Objects and Classes 17

Module 2 Lesson 2:Encapsulation 26

Module Assessment 33

Module Summary 34

References 35

MODULE 3: OBJECT-ORIENTED DESIGN PRINCIPLES 36

Module 3 Lesson 1:Abstraction 37

Module 3 Lesson 2:Inheritance 44

Module 3 Lesson 3:Polymorphism 50

Module Assessment 59

Module Summary 60

References 61

MODULE 4: FILE HANDLING AND DATABASE CONNECTIVITY 62

Module 4 Lesson 1:File Input/Output Streams 63

Module 4 Lesson 2:Database Connectivity 71

Module Assessment 84

Module Summary 85

References 86

Appendices.....87
Appendix A: Grammarian Certificate.....88
Appendix B: Course Syllabus.....89

MODULE 01:
Introduction to Object-Oriented Programming

Hello students! Welcome to Module 1! You're officially starting your journey into Object-Oriented Programming, and this is where it all begins! In this module, you'll be introduced to the fundamental concepts of OOP, helping you understand the nature of objects and the four essential features that define OOP. You'll also get to know Java, as a popular OOP language and the fundamental tokens used in Java code. Don't worry, everything is explained step by step to help you feel confident as you move forward.

This module is divided into two easy-to follow lessons:

Lesson 1: Procedural vs Object-Oriented Approach
Lesson 2: The Building Blocks of Java: A Review of Fundamental Concepts

At the completion of this module, you will be able to:

1. Compare and contrast the procedural approach with the object-oriented programming approach.

Ready to begin? Let's dive into the lessons and start building your OOP foundation!

Module 1 Lesson 1

Procedural vs. Object-Oriented Programming

WEEK 2

LEARNING OUTCOMES

1. Analyze the key characteristics, advantages, and limitations of procedural and object-oriented programming.
2. Demonstrate appreciation for the principles of both approaches by recognizing their appropriate use in real-world scenarios.
3. Construct simple code examples that demonstrate both procedural and object-oriented approaches.

INTRODUCTION

Dear students, are you ready to dive into object-oriented programming? In this lesson, we will explore the concept of objects and learn how to utilize them to create programs that simulate real-world scenarios.



ACTIVITY

Write a Java program that adds two numbers. Afterward, answer the following questions below.



ANALYSIS

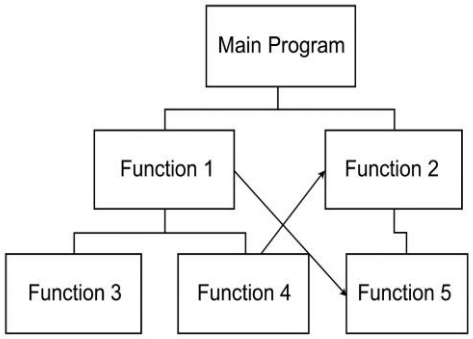
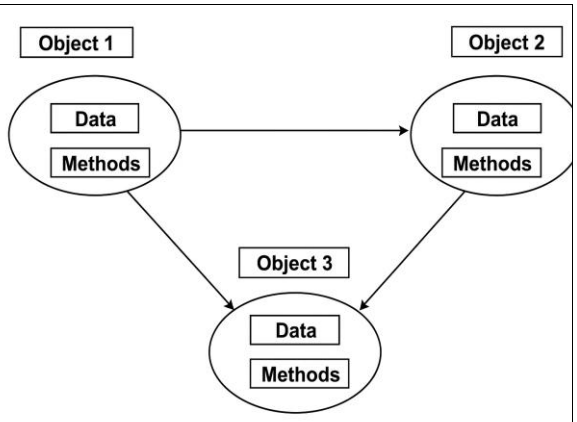
1. What is the purpose of the ***import*** statement in Java?
2. Why is the ***main*** method essential in a class?



ABSTRACTION

Programming Approaches

Procedural Approach	Object-Oriented Approach
• Used in older languages • "a set of procedures" or a "set of functions" • Functions are called repeatedly to execute tasks that they perform.	• Used in newer languages • Objects of the same class have the same data elements and methods • Objects send and receive messages to invoke actions • A collection of objects that interact.

• Every module has its data and function, which other modules can call.  **Procedural Approach**	 **Object-Oriented Approach**

Scenario 1. Add a bus form that records the following information: *color, engine size, transmission type, and number of passengers*.	
Procedural. recreate the entire form, repeating the code for *color, engine size, and transmission type*.	**OOP.** Extend the *vehicle class* with a *bus class* and add a *method* to calculate the number of passengers.

Scenario 2. The client wants the *color* emailed to him.	
Procedural. Change the different forms: *cars, trucks, and bus* to email the *color* to the client rather than storing it in the database.	**OOP.** Change the *color method* in the *vehicle class* because the c*ar, truck, and bus classes* all extend (or inherit from) the *vehicle class*; therefore, they are automatically updated.

Scenario 3: Found a bug in the *transmission type* and need to fix it.	
Procedural. Open and update each form.	**OOP.** Fix the *transmission method* in the *vehicle class*, and all classes that inherit from it will be changed.

Object-oriented Programming (OOP) is a programming approach focused on the concept of "objects". It contains data in the form of fields called *attributes or properties* and *methods*. Its primary purpose is to make programs more flexible and maintainable. OOP brings data together with its behavior (methods) in a single location (object) to help us understand how a program works.

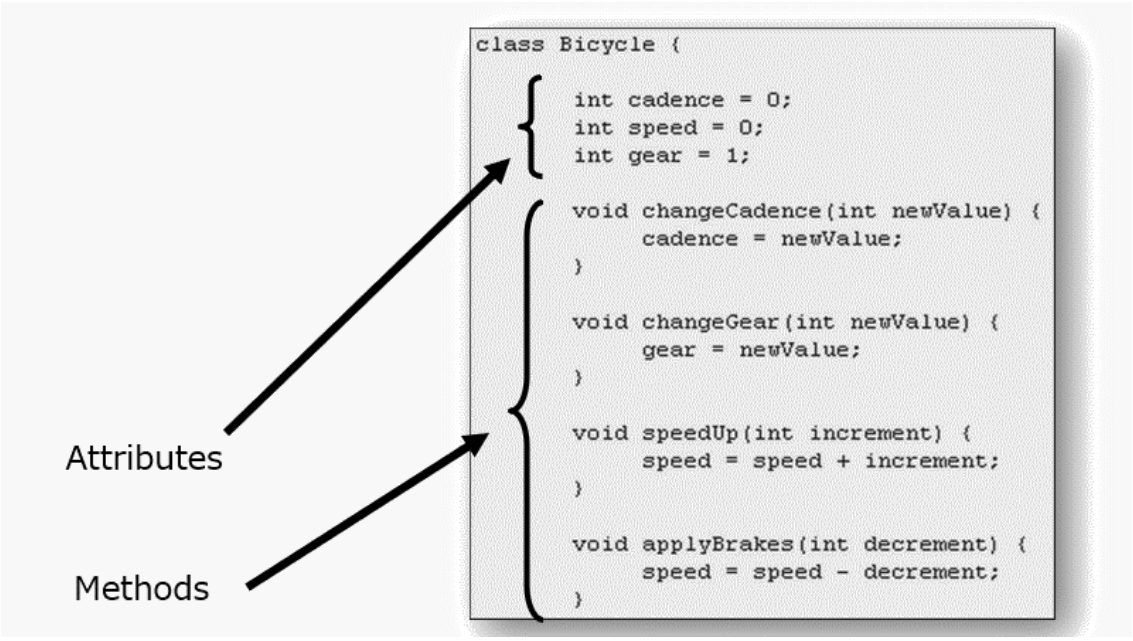
Basic Terminologies

Object is anything that can be represented by data in a computer's memory and manipulated by a computer program is referred to as data. Usually, a person, place or thing.

Attribute. Description of objects in a class (properties).

Method. An action performed by an object.

Class. A group of objects with the same *properties* (*attributes*) and *methods*. Category or similar objects



Advantages of Object-oriented Approach

- **Modularity.** An object's source code can be developed independently and reused.
- **Information-hiding.** Details of object's internal implementation remain hidden from the outside world.
- **Code reuse.** If an object already exists, you can use it in your program. Experts can build and test these objects, so you can rely on them to work correctly in your code.
- **Pluggability and debugging ease.** If an object causes issues in your program, you can replace it for another one.

Object-Oriented Design Principles

Abstraction is a process that only shows relevant data and hide unnecessary information to the users.

Encapsulation is the concept used to describe how the interface is separated from the implementation. You can think of encapsulation as "data-hiding", allowing for the exposure of certain parts of an object while other parts remain hidden.

Inheritance is a process by which an object acquires the properties of another object. It is a way of organizing classes.

Polymorphism means objects are handled differently depending on their data type. Polymorphism means "multiple forms", which means that for a certain class of action, there is one method with multiple implementations.



APPLICATION

Instructions: Read each sentence carefully and write the correct answer.

1. In a procedural approach, functions are called repeatedly to execute tasks in an object-Oriented approach _____ are called to perform actions or tasks.
2. It is a group of objects with the same properties and the same methods. In short, it is a category of similar objects.
3. It is an OOP design principle that is also known as data hiding, where the interface is separated from the implementation.
4. It is a process by which an object acquires the properties of another object.
5. These are the descriptions of objects in a class.
6. In this programming approach, modules are self-contained units with their own data and functions, allowing them to be called by other modules.
7. It is a keyword that tells the compiler to invoke a certain package.
8. In OOP, this refers to a person, place, or thing.
9. It is a process in which you only show the user relevant data and hide unnecessary information.
10. This programming approach breaks the program into entities such as objects or classes.
11. This principle is known to handle objects differently based on their data type. It allows for a single method to have multiple implementations for different classes of objects.
12. -15. What are the advantages of the object-oriented approach?

Well done students ! You've successfully completed this lesson and get ready for the next exciting challenge ahead.

Module 1 Lesson 2

The Building blocks of Java: A Review of Fundamentals Concept

WEEK 3 TO WEEK 5

LEARNING OUTCOMES

1. Explain and apply the fundamental concepts of Java including tokens, control structures, and error types in simple code.
2. Show a positive attitude toward writing clean and error-free Java code
3. Write and debug Java programs using proper syntax, control flow and error handling.

INTRODUCTION

Welcome to Module 1, Lesson 2, dear students. In this lesson, we will delve into the fundamental elements of Java programming. We will explore Java tokens, understand how conditional statements and loops control program flow, and discuss the different types of errors, effective debugging techniques, and best practices for writing error-free code.



ACTIVITY

Reflect on the Java code and program flow from the activity in Lesson 1, then answer the following questions.



ANALYSIS

1. How are variables defined in Java?
2. What Java tokens were used in the program?
3. Were there any difficulties you faced while coding? If so, can you describe the specific errors you encountered?



ABSTRACTION

A **Programming Language** is a set of rules, symbols, and special words used to construct programs.

Syntax rules tell you which statements are legal, or accepted by the programming language, and which are valid or not valid.

Semantic rules determine the meaning of instructions.

A **token is** the smallest individual unit of a program written in any programming language. Java's tokens are divided into three categories: *special symbols, reserved words, and identifiers*.

Special Symbols

Mathematical symbols: *addition, subtraction, multiplication, and division*	+ - * /
Punctuation marks from English grammar	. ; ? ,
Separate items in a list	(,) *commas*
End a java statement	(;) *semicolons*
Combine two or more Strings *(Concatenation)*	+
Special symbol	( ) *blank*
used in Boolean expressions: *less than or equal to, not equal, equal, greater than or equal to*	<= != == >=

Reserved words are also called keywords. Letters in a reserved word are always lowercase. It cannot be redefined within any program and cannot be used for anything other than their intended use.
Some reserved words: *int, float, double, char, void, public, static, throws, return*

Identifiers are names of things, such as variables, constants, and methods, that appear in programs.

1. Predefined identifiers
    - generating output: *print, println, printf*
    - input data: *nextInt, nextDouble, next, nextLine*
2. Identifiers defined by the user
    - letters, digits, underscore character (_), dollar sign ($),
    - Must begin with a letter, underscore, or dollar sign
    - No other symbols are permitted to form an identifier.
    - Legal identifiers: *first, conversion, payrate, counter2, $Amount*
    - Illegal identifiers: *employee Salary, Hello!, One+two, 2nd*

A data type is a set of values, along with a set of operations that can be performed on those values.

Primitive data types are the fundamental data types in Java.

1. **Integral** – a data type that deals with integers, or numbers without a decimal point

int	• Integers in Java are numbers. • Positive integers do not require a + sign in front of them. • No commas are used within an integer.
char	• Represent single characters, letters, digits, and special symbols. • A blank space is a character. • When using char, enclose each character represented within single quotation marks (' '). • Examples: 'A', 'a', '2', '*', '+', '$', '&', ' ' • Allows only one symbol to be placed between the single quotation marks. Thus, 'abc' is not of type char.
byte	• smallest data type, used in array
long	• can hold the largest integer values. It is useful for storing numbers that outgrow the integer data type.

2. **Floating-point** is a data type that deals with decimal numbers. By default, floating-point numbers are considered of type double
   - *float* - single precision (number of decimal places is 6 or 7)
   - *double* - double precision (number of decimal places) is typically 15

3. **Boolean** is a data type that deals with logical values (true or false). The primary purpose of this data type is to manipulate logical (Boolean) expressions.

A String is a sequence of zero or more characters. Are enclosed in double quotation marks (" "). We process strings as single units. Null or empty strings are strings that contain no characters. Storing data in the computer's memory is a two-step process.

1. Instruct the computer to allocate memory
2. Include statements in the program to put the data into the allocated memory.

Allocating Memory

Named constant is a memory location whose content is not allowed to change during program execution. The reserved word *final* specifies that the value stored in the identifier is fixed and cannot be changed.

Syntax: *final dataType IDENTIFIER = value;*
Example: *final double PI = 3.1416;*

Variable is a memory location whose content may change during program execution.

Syntax: *dataType identifier1;*
Example: *int num1;*

Putting Data into Variables

1. **Use an assignment statement .** A variable is said to be initialized the first time a value is assigned to it. In java, = *(equal sign)* is called the assignment operator.
2. **Use input(read) statements** – *using nextInt(), nextDouble(), next(), nextLine()*

Control Structures

A computer can process a program in one of three ways:

1. **Sequence**
2. **Selection** - altering the flow of program execution by selecting or choice, which is also called a *branch*.
3. **Repetition** - altering the flow of program execution by the repetition of statement(s) over and over again using a structure called *loop*.

For loop allows you to execute a block of code repeatedly for a specified number of times. It is often used when you know in advance how many times you need to repeat a task.

Syntax:
for (initialization; condition; update) {
Statement;
}

- **Initialization:** Executed once before the loop starts. Used to declare and initialize a loop counter variable.
- **Condition:** Evaluated before each iteration. If it is true, the loop body is executed. If it is false, the loop terminates.
- **Update:** Executed after each iteration. It is usually used to update the loop counter variable.

Example 1. Display the numbers 60,50,40,30,20,10.

```
int x=0;


for(x=50;x>=1;x-=10 )
System.out.println(x);
```

Example 2. Display the square root of 25,20,15, and 10.

```
int number;

for(number=25;number>=10;number-=5)
    System.out.println("the square root of " + number+ " is" + Math.sqrt(number));
```

Example 3. Display the sum of the integers between a starting value and an ending value, using a designated increment.

```
   int cntr,sum,startingValue,endingValue,increment;

   startingValue=10;
   endingValue=20;
   increment=3;


   sum=0;
   for(cntr=startingValue;cntr<=endingValue;cntr+=increment)
       sum +=cntr;
   System.out.println(sum);
```

A **while loop** repeatedly executes a block of code as long as a specified condition remains true. It is often used when the exact number of iterations needed is not known beforehand.

Syntax:

```
Counter = 0;              //initialization
while (counter < N) {     //condition
  Statements;
  Counter++;              //update
```

```
    int i=0;
    while (i<=20){
        System.out.println( i + "");
        i = i+5;
    }
}
```

A **do-while loop** executes a block of code at least once and then continues to repeat as long as a specified condition remains true. It is similar to a while loop, but the condition is checked after the loop body is executed.

Syntax:

```
Counter = 0;                    //initialization
do{
    Statements;
    Counter++;                  //update
}while (counter < N)            //condition
```

```
int i=50;
 do{
      System.out.println( i + "");
      i = i-5;
 }while (i>5);
```

An **if-else statement** is a decision-making control flow statement that allows your program to execute different code blocks based on a specified condition.

Syntax:

```
if (condition) {
      statement;
 } else {
      statement;
}
```

```
int number=25;

if(number>0){
    System.out.println("positive");
}else{
     System.out.println("negative or zero");
}
}
```

A **nested if-else statement** is an if-else statement that is contained within another if-else statement. This allows you to make more complex decisions and handle multiple conditions.

Syntax:

```
if (condition) {
      statement;
 } else if {
      statement;
} else {
     statement;
}
```

```
int grade = 90;

if (grade >= 90) {
    System.out.println("Grade: Outstanding");
} else if (grade >= 80) {
    System.out.println("Grade: Satisfactory");
} else if (grade >= 70) {
    System.out.println("Grade: Poor");
} else {
    System.out.println("Grade: Failed");
}
}
```

A switch statement is a control flow statement used to efficiently select one of several code blocks to execute based on the value of an expression. It is often used as an alternative to chained if-else statements when you have multiple conditions to check.

Syntax:

```
switch (expression) {
     case value1:
              statement;
              break;
 case value2:
             statement;
              break;
   ...
              default:
              statement;
}
```

```
      int dayOfWeek = 3;

switch (dayOfWeek) {
case 1:
     System.out.println("Monday");
     break;
case 2:
     System.out.println("Tuesday");
     break;
case 3:
     System.out.println("Wednesday");
     break;
default:
     System.out.println("Invalid day");
  }
}
```

Programming Errors

Syntax errors occur when we violate a syntax rule, no matter how minor it is.

Run-time errors occur when we ask the computer to perform an action it considers illegal. Dividing by 0, x/y.

Logic errors are also known as design errors or bugs. It occurs when we fail to express ourselves accurately. The instructions do not do what we intended, and thus logically incorrect.



APPLICATION

Instructions: Write a program that computes a person's net income. The user inputs are the gross income and number of dependents.
Fixed tax rate of 20%

- There is a P10,000 standard deduction
- There is a P 2,000 additional deduction for each dependent
- Gross income must be entered
- The income is expressed as decimal number
- Gross income (income without deduction and taxes)
- Net income (income after the deductions including tax)

Great work students ! You've successfully completed Module 1, be proud and get ready to keep the momentum going in Module 2!

MODULE ASSESSMENT

HANDS-ON PROGAMMING

1. Write a program to determine an employee's salary based on the number of hours worked. The program should calculate regular pay and overtime pay separately, using a base salary of 25,000 and an overtime rate of 250.50 per hour.
2. Create the following grid of numbers using a for loop.

   1 2 3 4 5
   2 3 4 5 6
   3 4 5 6 7
   4 5 6 7 8
   5 6 7 8 9

3. Create a program that generates a sequence of numbers from a user-specified starting point to a user-specified ending point, in ascending order. Use both while and do-while loops to execute the program.
4. Write a program that utilizes a switch statement to allow users to select items from a predefined menu. The program should prompt users for the desired quantity of each selected item. Once the user has finished their order, the program should calculate and display the total bill.

MODULE SUMMARY

This module covers two main approaches to programming: Procedural and Object-Oriented (OOP).

Procedural programming, as used in older languages such as C, VB, Fortran, and Pascal, relies on a set of procedures or functions to perform tasks. These functions are repeatedly called within a program to execute specific actions. This approach focuses on breaking down the program based on its functions, emphasizing actions and verbs.

Object-oriented programming (OOP), used in modern languages like C++, Java, and C#, focuses on building programs as collections of interacting objects. These objects bundle data (attributes) and actions (methods) together, mimicking real-world entities. Objects communicate by sending messages to each other, invoking specific methods. This approach emphasizes the "who" (objects) rather than the "what" (actions) of the program.

OOP offers several advantages over procedural programming. OOP promotes modularity by allowing objects to be developed and maintained independently of one another. It also enforces information hiding, shielding the internal workings of an object from external interference. Code reusability is enhanced by the ability to reuse existing objects in new programs. Additionally, OOP facilitates easier debugging and maintenance, as problematic objects can be replaced or modified with minimal impact on the overall system.

OOP centers around the concept of objects, which represent real-world entities. Objects possess data attributes, which define their characteristics, and methods, which describe their behaviors. Classes serve as blueprints for creating objects, defining their shared attributes and methods. This approach promotes code reusability, modularity, and flexibility in software development.

OOP incorporates several key principles to enhance code organization and maintainability. Abstraction emphasizes focusing on the essential features of an object, hiding unnecessary complexity. Encapsulation promotes information hiding, separating the object's interface from its implementation. Inheritance enables code reuse by allowing the creation of new classes based on existing ones. Polymorphism enables objects of different types to be treated uniformly, promoting flexibility and adaptability in software design.

REFERENCES

Burd, B. (2021). *Beginning programming with java for dummies*. John Wiley & Sons, Inc.

Budd, T.A. (2021). *An Introduction to object-oriented programming*. Addison Wesley.

Sarcar, V. (2020). *Interactive object-oriented programming in java*. Springer Science.

3G E-Learning. (2023). *Object-oriented Programming*.

MODULE 02:
Object-Oriented Programming in Java

Welcome to Module 2, dear students! Now that you've learned the basics of Object-Oriented Programming, it's time to bring those ideas to life using Java. This module will guide you through the process of designing and creating object-oriented programs in Java. You'll explore core concepts like objects, classes, constructors, and methods. Additionally, in this module, you'll learn how to apply the principles of encapsulation in Java Programming. Let us get started!

This module is divided into two lessons:

Lesson 1: Objects and Classes
Lesson 2: Encapsulation

At the completion of this module, you will be able to:

1. Design, implement, test, and debug programs using OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.

Ready to level up your Java skills? Let's get started and jump into the lessons!

Module 2 Lesson 1

Objects and Classes

WEEK 6

LEARNING OUTCOMES

1. Describe and apply the concepts of objects and classes in Java.
2. Demonstrate interest and confidence in applying object-oriented thinking to code design.
3. Develop Java programs that define and use classes and objects to model real-world entities.

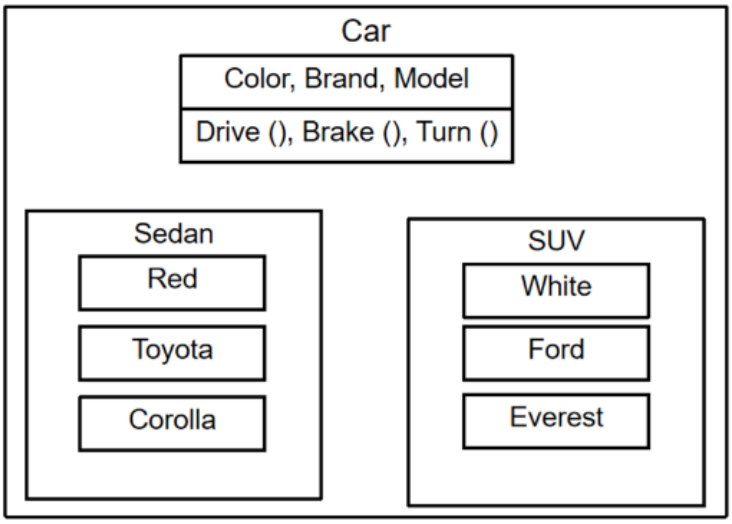
INTRODUCTION

Dear students, welcome to Module 2, Lesson 1. In this lesson, you will learn about the fundamental relationship between objects and classes. And understand the essential role of constructors in object creation. By the end of this lesson, you will be able to create Java programs that effectively utilize multiple classes.



ACTIVITY

Examine the diagram carefully and answer the following questions.





ANALYSIS

1. Determine the classes and objects represented in the figure.
2. In your own words, explain how you believe classes and objects are connected or related within this context.



ABSTRACTION

A **class** is like a blueprint that defines the structure and behavior of objects. It specifies the *attributes* and *methods* that objects of that class will have. An **object** is an instance of a class. It represents a real-world entity or concept. When you create an object from a class, you are essentially creating a specific instance of that class. **Driver Class** contains the *main()* method. **main() Method** is the starting point of the Java Virtual Machine (JVM). The JVM starts executing the code in the main() method.

Driver Class

```
public class Driver {
    public static void main(String [] args) {
        Person aPerson = new Person();
        aPerson.sayHello();
    }
}
```

Class Person

```
public class Person {
    public void sayHello()
        System.out.println("I don't wanna say hello.");
    }
}
```

Attributes can be variable, or constant and are almost always set to *private*. Set the *access modifier* on all your methods to *public*.

Example:

```
public class Person {
    private int age;
}
```

Example:

```
public class Person {
    //method definition
    private void sayAge(){
        System.out.println("My age is" + age);
    }
```

Access Modifier in Java control the visibility of classes, methods, and variables within a program. They determine which parts of your code can access these elements.

1. **public:** can be accessed from anywhere within the program, including other classes in different packages (*most accessible).*
2. **private:** can only be accessed within the same class *(least accessible*).
3. **protected:** can be accessed within the same class, its subclasses, and default other classes in the same package.
4. **default:** If no access modifier is specified, the element has default access. This means it can be accessed within the same class and other classes in the same package *(no keyword).*

Parameter Passing is the mechanism that pass values to a method when it is called. These values are used within the function to perform calculations or modify data.

1. **Pass by Value:** A copy of the value is passed to the function. Any changes made to the parameter within the function do not affect the original value outside the function. This is the default method for passing primitive data types
2. **Pass by Reference:** A reference to the object is passed to the function. Changes made to the object within the function can affect the original object outside the function. This is used for passing objects and arrays.

```
    public static void main(String args[]){

        int num=10;
        String str = "Hello";


        str=changeReference(str);
        num=changeValue(num);


        //System.out.println("num after change: " + x);
         System.out.println("after change: " + num);
        System.out.println("after change: " + str);

    }

    public static int changeValue(int x){
         x=20;
       return x;

    }

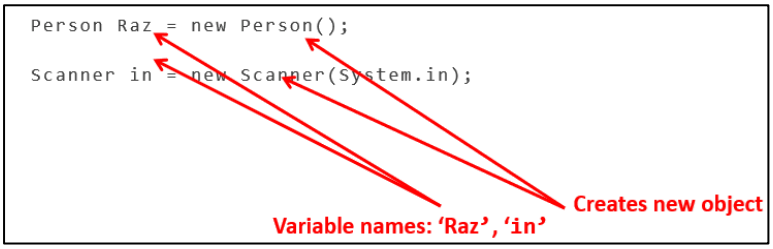
    public static String changeReference(String s){
         s="World";
         return s;
    }

}
```

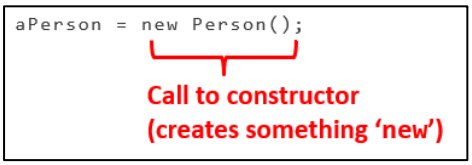
Return Type specifies the data type of the value that a method will return to the calling code.

Methods are behaviors or actions for each instance of a class.

Instantiation is creating a new instance of a class. Instances of a class are referred to as objects.



Constructor is a special method to initialize the attributes of an object as the objects are instantiated (created). Constructors can take parameters but never have a return type. The constructor is automatically invoked whenever an instance of the class is created.



Example: Adds an int y parameter to the constructor. Inside the constructor we set x to y (x=y). When we call the constructor, we pass a parameter to the constructor (5), which sets the value of x to 5.

```
public class Main {
    int x;

    public Main(int y){  // constructor
     x=y;
    }


     public static void main(String[] args){
      Main obj = new Main(5);
      System.out.println(obj.x);
    }
  }
```

Constructor with many Parameters

```
public class Main {
    String StudentCourse;
    int Studentyear;

    public Main(String course,int year){  // constructor
     StudentCourse=course;
     Studentyear=year;
    }


     public static void main(String[] args){
      Main obj = new Main("BSIT",2);
      System.out.println("Course: " + obj.StudentCourse + " ,Year: " + obj.Studentyear );
    }
  }
```

Calling Methods

```
Example:

   // calling Scanner class method

     Scanner in = new Scanner(System.in);

     System.out.print("Enter your age: ");

     age = in.nextInt();
```

Scanner variable

Calling method

Calling a method outside the body of the class (in another class definition).

```
Example:                                          raz.becomeOlder()

public class Driver{

  Person raz = new Person();

  Person vern = new Person();
```

Calling a method inside the body of the class (where the method has been defined). You can just directly refer to the method (or attribute)

```
public class Person{
   private int age;

     public void birthday() {
        becomeOlder();  // access a method
   }

     public void becomeOlder() {
        age++;    // access an attribute
 }
```

Creating An Object

The keyword ***new*** calls the constructor to create a new object in memory

Create a variable that refers to an object

Person raz;

Create a *new* object

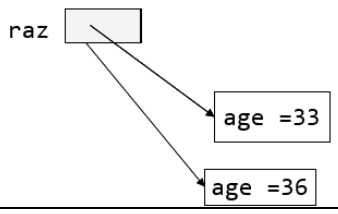
raz = new Person();

```
Person raz;

  raz = new Person(33);

  raz = new Person(36);
```

raz is a reference to a Person object



Viewing And Modifying Attributes

Accessor methods: *get() method* is used to determine the current value of an attribute

```
public int getAge()
  {
     return(age);
  }
```

Mutator methods: *set() method* is used to change an attribute (set it to a new value)

```
public void setAge(int anAge)
  {
     age = anAge;
  }
```

Creating an object from a class : An object called myObj and print the value of x.

```
public class Main {

  int x = 2;

  public static void main(String[] args){
      Main myObj = new Main();
      System.out.println(myObj.x);
  }

}
```

Create multiple objects of one class : Two objects of class Main

```
public class Main {

  int x = 2;

  public static void main(String[] args){
      Main myObj1 = new Main();
      Main myObj2 = new Main();
      System.out.println(myObj1.x);
      System.out.println(myObj2.x);
  }

}
```

Multiple Classes : You can create an object of a class and access it in another class. This is often used for better organization of classes: one class has all the attributes and methods. Other classes hold the main() method or the code to be executed

Main.java

```
public class Main {

  int x = 2;

}
```

Second.java

```
public class Second {

    public static void main(String[] args){
        Main myObj = new Main();
        System.out.println(myObj.x);
    }
}
```

Attributes are variables within a class. A class with two attributes

```
public class Main {

  int x = 2;
  int y = 6;
  }
```

Create an object and print the value of the attributes

```
public class Main {

  int x = 2;
  int y = 6;

  public static void main(String[] args){
      Main obj=new Main();
      System.out.println(obj.x);
      System.out.println(obj.y);
  }
  }
```

Modify Attributes : Modify the value of x to 25;

```
public class Main {
  int x = 2;

    public static void main(String[] args){
      Main obj=new Main();
      obj.x=25;
      System.out.println(obj.x);

  }
  }
```

Multiple objects - If you create multiple objects of one class, you can change the attribute values in one object, without affecting the attribute values in the other:

Example: Change the value of x to 25 in myObj2, and leave x in myObj1 unchanged

```
public class Main {
  int x = 2;

    public static void main(String[] args){
      Main obj1=new Main();
      Main obj2=new Main();
      obj2.x=25;
      System.out.println(obj1.x);
      System.out.println(obj2.x);

  }
  }
```

Multiple attributes: Create attributes for first name, last name, and age, then display these attributes.

```
public class Main {
  String fname = "razel";
  String lname ="aguilar";
  int age=35;

    public static void main(String[] args){
      Main obj=new Main();
      System.out.println("Name: " + obj.fname + " " + obj.lname);
      System.out.println("Age: " + obj.age);

  }
  }
```

Methods are declared within a class and are used to perform specific actions. Create a method that will display “hello world”

```
public class Main {


    static void mymethod(){
        System.out.println("Hello world");
    }

     public static void main(String[] args){
      mymethod();
    }
  }
```

A static method can be accessed without creating an object of the class.

```
public class Main {


    static void mymethod(){
        System.out.println("Hello world");
    }

     public static void main(String[] args){
      mymethod();
    }
  }
```

public method can only be accessed by objects.

```java
public class Main {


   public void mymethod(){
        System.out.println("Hello world");
    }

     public static void main(String[] args){
      Main obj = new Main();
      obj.mymethod();
    }
  }
```

An abstract method belongs to an abstract class and does not have a body. The subclass provides the body.

```java
abstract class Animal {
    public abstract void makeSound(); // Abstract method
}

class Dog extends Animal {
    public void makeSound() {
        System.out.println("Woof!");
    }
}
```

Access method with an object

```java
public class Main {


  public void fullThrottle(){
       System.out.println("Car is going fast");
   }

  public void speed(int maxSpeed){
       System.out.println("speed " + maxSpeed);
  }

    public static void main(String[] args){
     Main obj = new Main();
     obj.fullThrottle();
     obj.speed(300);
   }
 }
```



APPLICATION

Create an object-oriented Java program that calculates and displays a person's age based on their birth year.

Well done ! You've successfully mastered this lesson, excellent work! Keep moving forward and continue building on your skills. You're doing great!.

Module 2 Lesson 2

Encapsulation

WEEK 7 TO WEEK 8

LEARNING OUTCOMES

1. Explain the concept of encapsulation and apply access modifiers to control data visibility and access in Java classes.
2. Recognize the importance of encapsulation in promoting data security.
3. Implement encapsulation in Java by defining attributes as private and providing appropriate getter and setter methods.

INTRODUCTION

Dear students, welcome to Module 2, Lesson 2! In this lesson, let's explore encapsulation and learn how to apply it effectively in your OOP projects. We'll cover practical examples and best practices to help you write better code.



ACTIVITY

Examine the given *Main class* and provide answers to the questions given.

```
public class Main {

  int x = 2;

}
```



ANALYSIS

1. What is the access modifier used?
2. Can you instantiate multiple instances of the same class? (create multiple objects of one class)
3. Demonstrate how to instantiate two separate instances of the Main class.



ABSTRACTION

Encapsulation in Java is a process of wrapping code and data together into a single unit, for example, a capsule that contains several medicines. The meaning of encapsulation is to make that "sensitive" data hidden from users.

We can create a fully encapsulated class in Java by making all the data members of the class private.

Declare class variables or attributes as private. Private variables can only be accessed within the same class (an outside class has no access to it). However, it is possible to access them if we provide public *get and set methods*. Public get and set methods to access and update the value of a private variable. The ***get method()*** returns the variable value. While the ***set method()*** sets the value.

Advantages of Encapsulation

- **Class read-only or write-only.** Providing only a setter or getter method, you can make the class property accessible.
- **Provide control over the data**. Suppose you want to set the value of id to be greater than 100 only; you can write the logic inside the setter method. You can write the logic to prevent storing negative numbers in the setter methods.
- **Data hiding.** Other classes will not be able to access the data through the private data members.
- **Easy to test.** It is better for unit testing.
- The standard IDEs provide the facility to generate getters and setters. Therefore, creating an encapsulated class in Java is easy and fast.

class Person

```
public class person {

    private String name; //private=restricted acess

    //getter
    public String getName(){
        return name;
    }

    //Setter
    public void setName(String newName){
        this.name=newName;
    }
}
```

class Main (Driver class)

```
public class Main {

     public static void main(String[] args){
     person p = new person();
     p.setName("North"); // set the value of the name variable to "North"
     System.out.println(p.getName());
  }
}
```

get() method is used to determine the current value of an attribute

```
public int getAge()
  {
     return(age);
  }
```

set() method is used to change an attribute (set it to a new value)

```
public void setAge(int anAge)
  {
     age = anAge;
  }
```

Example 1:

```
public class person {

    private String name; //private=restricted acess

    //getter
    public String getName(){
        return name;
    }

    //Setter
    public void setName(String newName){
        this.name=newName;
    }
}
```

```
public class Main {

    public static void main(String[] args){
    person p = new person();
    p.setName("North"); // set the value of the name variable to "North"
    System.out.println(p.getName());
  }
}
```

Example 2: Create an object-oriented Java program that calculates and displays a person's age based on their birth year in Java

```java
public class Person {

      private int birthYear;
      private int currentYear=2024;
    public Person(int birthYear) {
        this.birthYear = birthYear;
    }

    public int calculateAge() {

        return currentYear - birthYear;
    }
}
```

```java
public class AgeCalculator {

    public static void main(String[] args) {
        int birthYear = 1988;
        Person person = new Person(birthYear);
        int age = person.calculateAge();
        System.out.println("Your age is: " + age);
    }
```

Example 3: Design a fully encapsulated Account class that can show the account number, owner's name, and balance.

```java
class Account {
    private String accountNumber;
    private String ownerName;
    private double balance;

    //constructor
    public Account(String accountNumber, String ownerName, double balance) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
    }

     // Getter and setter for account number
    public String getAccountNumber() {
        return accountNumber;
    }
```

```
    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    // Getter and setter for owner's name
    public String getOwnerName() {
        return ownerName;
    }

    public void setOwnerName(String ownerName) {
        this.ownerName = ownerName;
    }


    // Getter and setter for balance
    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

       // Method to display account details
    public void displayAccountDetails() {
        System.out.println("Account Number: " + getAccountNumber());
        System.out.println("Owner's Name: " + getOwnerName());
        System.out.println("Account Balance: " + getBalance());
```

```
class AccountTest {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt user to input account details
        System.out.print("Enter Account Number: ");
        String accountNumber = scanner.nextLine();

        System.out.print("Enter Owner's Name: ");
        String ownerName = scanner.nextLine();

        System.out.print("Enter Account Balance: ");
        double balance = scanner.nextDouble();

        // Create Account object using user inputs
        Account account = new Account(accountNumber, ownerName, balance);

        // Display account details
        System.out.println("\nAccount Information:");
        account.displayAccountDetails();
```

Example 4: Create a Java program demonstrating encapsulation to calculate the discounted price of an item.

```java
public class Discount {
    private double originalPrice;
    private double discountPercentage;

     // Constructor
    public Discount(double originalPrice, double discountPercentage) {
        this.originalPrice = originalPrice;
        this.discountPercentage = discountPercentage;
    }

    // Getter and setter for original price
    public double getOriginalPrice() {
        return originalPrice;
    }

    public void setOriginalPrice(double originalPrice) {
        this.originalPrice = originalPrice;
    }

     // Getter and setter for discount percentage
    public double getDiscountPercentage() {
        return discountPercentage;
    }

    public void setDiscountPercentage(double discountPercentage) {
        this.discountPercentage = discountPercentage;
    }

     // Method to calculate the discounted price
    public double calculateDiscountedPrice() {
        return originalPrice - (originalPrice * discountPercentage / 100);
    }
```

```java
class DiscountCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt user to input item details
        System.out.print("Enter the original price of the item: ");
        double originalPrice = scanner.nextDouble();

        System.out.print("Enter the discount percentage: ");
        double discountPercentage = scanner.nextDouble();

        // Create Item object using user inputs
        Discount d = new Discount(originalPrice, discountPercentage);

        // Calculate the discounted price
        double discountedPrice = d.calculateDiscountedPrice();

        // Display the discounted price
        System.out.println("\nOriginal Price: P" + d.getOriginalPrice());
        System.out.println("Discount Percentage: " + d.getDiscountPercentage() + "%");
        System.out.println("Discounted Price: P" + discountedPrice);

        scanner.close();
    }
```



APPLICATION

Design an executable class featuring a constructor that takes at least four arguments (parameters). The parameters should be a combination of strings and integers.

Great job, students! You've completed this lesson with excellence, so keep the momentum going and continue sharpening your skills.

MODULE ASSESSMENT

1. Design a fully encapsulated Account class that can show the account number, owner's name, and balance.
2. Design a Java class implementing encapsulation to calculate the area of square securely.

MODULE SUMMARY

In object-oriented programming, classes act as blueprints for creating objects. A class defines the attributes (data) and methods (functions) that objects of that class will have. Objects are instances of classes, representing real-world entities.

Java programs are typically structured with a driver class that contains the main() method, which serves as the program's entry point. Inside a class, you can define attributes to store data and methods to perform actions on that data. Access modifiers control the visibility of these elements, allowing you to restrict access from other parts of the program.

When passing data to methods (functions), you can either pass by value (a copy) or by reference (a reference to the original object). Methods can also have a return type to specify the data they return. Constructors are special methods that initialize an object's attributes when it is created.

You can call methods directly within a class or by using an object of the class outside the class. Creating objects involves using the' new' keyword to invoke the constructor. Methods differ from functions as they are invoked through objects, while functions can be called independently.

Encapsulation is a concept where you bundle data and the code that manipulates it together. This is often achieved by making attributes private and providing public getter and setter methods for controlled access. Encapsulation offers several advantages, including data protection, improved code organization, and easier testing.

REFERENCES

Burd, B. (2021). *Beginning programming with java for dummies*. John Wiley & Sons, Inc.

Budd, T.A. (2021). *An Introduction to object-oriented programming*. Addison Wesley.

Sarcar, V. (2020). *Interactive object-oriented programming in java*. Springer Science.

3G E-Learning. (2023). *Object-oriented Programming*.

MODULE 03:
Object-Oriented Design Principles

Welcome to Module 3, dear students! You're now entering an exciting stage of your Object-Oriented Programming journey. In this module, we'll dive into the core design principles of OOP that make programs smarter, more flexible, and easier to maintain. This module delves into the fundamental design principles of object-oriented programming, including inheritance, polymorphism, and abstraction. These concepts build upon the foundation of classes and objects, enabling you to create more flexible, reusable, and maintainable code. Moreover, in this module, you'll gain the ability to solve real-world programming challenges by effectively applying abstraction, inheritance, and polymorphism.

This module is divided into two lessons:

Lesson 1: Abstraction
Lesson 2: Inheritance
Lesson 3: Polymorphism

At the completion of this module, you will be able to:

1. Design, implement, test, and debug programs using OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.

Ready to think like a software designer? Let's begin Module 3 and start coding smarter!

Module 3 Lesson 1

Abstraction

WEEK 10

LEARNING OUTCOMES

1. Explain the concept of abstraction and differentiate it from encapsulation, with examples of abstract classes and interfaces in Java.
2. Recognize the value of abstraction in simplifying complex systems.
3. Design and implement Java programs using abstract classes or interfaces.

INTRODUCTION

Hello students, welcome to Module 3, Lesson 1! In this lesson, we will dive into the world of abstraction. This powerful programming technique helps us simplify complex systems by focusing on what matters most and ignoring unnecessary details. We will explore different types of abstraction and learn how to use them effectively in our programming projects.



ACTIVITY

Abstraction is not just a programming concept; it's a fundamental principle that can be applied to real-world situations. Imagine a restaurant.



ANALYSIS

In a restaurant, what represents the abstract concept? Explain your answer.



ABSTRACTION

Abstraction is a process of hiding the implementation details and presenting only the functionality to the user. It shows only essential information to the user and hides internal details, for example, when sending an SMS, where you type and send the message. You are not aware of the internal processing involved in message delivery.

A class that is declared with the ***abstract*** keyword is known as an abstract class. It can have abstract and non-abstract methods. It needs to be extended, and its method needs to be implemented

Rules for Java Abstract Class

1. An abstract class must be declared with an *abstract* keyword.
2. It can have abstract and non-abstract methods. It cannot be instantiated.
3. It can have final methods.
4. It can have constructors and static methods also.

Abstract Class

```
abstract class A{

}
```

Abstract Method is a method that is declared abstract and does not have an implementation .

```
abstract void printStatus();

//no method body and abstract
```

Abstract Class with and Abstract Method

Bike is an abstract class that contains only one abstract method, *run()*. The *Honda* class provide its implementation.

```
abstract class Bike {
    abstract void run();
   }

class Honda extends Bike{
    void run(){
        System.out.println("running safety");
    }

    public static void main(String args[]){
        Bike obj = new Honda();
        obj.run();
    }
}
```

Abstract class can have a data member, abstract method, method body (non-abstract method), constructor, and even main() method.

Rule :

1. If there is an abstract method in a class, that class must be abstract.

2. If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract

Example 1:

```
abstract class Shape{
abstract void draw();
}


//in real scenario, implementation is provided by others i.e. unknown by end user

class Rectangle extends Shape{
 void draw(){
     System.out.println("drawing rectangle");
 }
}

 class Circle extends Shape{
 void draw(){
     System.out.println("drawing circle");}
 }


 //in real scenario, method is called by programmer or user
 class TestAbstraction1 {
     public static void main(String args[]){

      Shape s=new Circle();
      s.draw();
```

Example 2:

```
abstract class Bank{
abstract int getRateOfInterest();
}


class SBI extends Bank{
int getRateOfInterest(){
    return 7;
 }
}
class PNB extends Bank{
 int getRateOfInterest(){
     return 8;
 }
}

 class TestBank {
 public static void main(String args[]){
        Bank b;
        b=new SBI();
        System.out.println("Rate of Interest:" +b.getRateOfInterest()+ "%");
        b=new PNB();
        System.out.println("Rate of Interest:" +b.getRateOfInterest()+ "%");
     }
```

Example 3:

```
abstract class Bike2{
    Bike2(){
        System.out.println("bike is created");
    }
    abstract void run();
    void changeGear(){
        System.out.println("gear changed");

    }
}

//Creating a Child class which inherits Abstract class
class honda extends Bike2{
    void run(){
        System.out.println("running safely");
    }
}

//creating a Test class which calls abstract and non-abstract methods

class TestAbstraction2 {

   public static void main(String args[]){
        Bike2 obj=new honda();
        obj.run();
        obj.changeGear();
    }
}
```

Interface in Java

An **interface** in Java is a blueprint of a class. It has static constants and abstract methods. The interface in Java is a mechanism to achieve abstraction and multiple inheritance. There can only be abstract methods in a Java interface, not a method body. Interfaces can have abstract methods and variables. It cannot have a method body. Java Interface also represents the IS-A relationship.

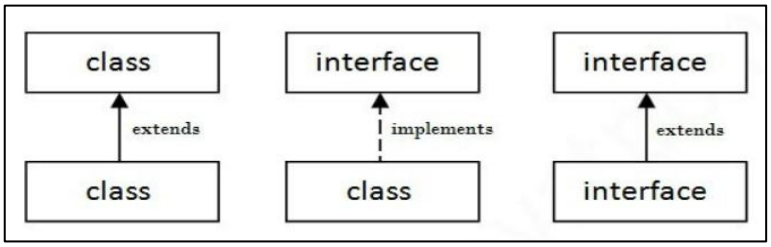
Why Use Java Interface

There are mainly three reasons to use interface. They are given below.

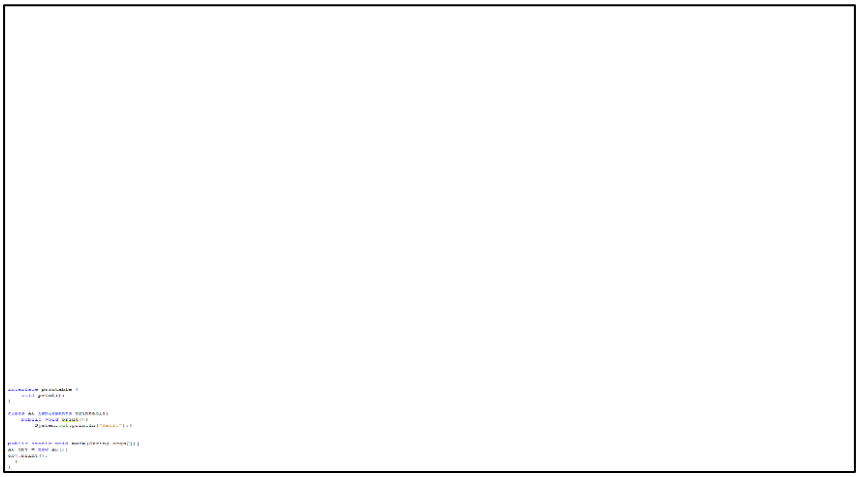
1. It is used to achieve abstraction.
2. By interface, we can support the functionality of multiple inheritance.
3. It can be used to achieve loose coupling.

An **interface** is declared by using the interface keyword. It provides total abstraction; this means that all methods in an interface are declared with an empty body, and all fields are public, static and final by default.
A class that implements an interface must implement all the methods declared in the interface.

Interface fields are **public, static and final by default**. Its methods are **public** and **abstract. A class extends another class, an interface extends another interface, but a class implements an interface.**



Example 1:



Example 2:

```
interface A{
     void a();
     void b();
     void c();
     void d();
}

abstract class B implements A{
     public void c(){
          System.out.println("I am c");
     }
}

class M extends B{
     public void a(){System.out.println("I am a");}
     public void b(){System.out.println("I am b");}
     public void d(){System.out.println("I am d");}
}


 class Test2 {
      public static void main(String args[]){
           A a=new M();
           a.a();
           a.b();
           a.c();
           a.d();
      }

}
```

Example 3:

```
interface Drawable{
    void draw();
}

//implementation: by second user
class Rectangle1 implements Drawable{
    public void draw(){
        System.out.println("drawing rectangle");
    }
}
    class Circle1 implements Drawable{
        public void draw(){
            System.out.println("drawing circle");
        }

}

//using interface: by third user
class TestInterface1 {
    public static void main(String[] args){
        Drawable d = new Circle1(); //In real scenario, object is provided by method  getDrawable()
        d.draw();
    }
}
```

A **class implements an interface**, but one **interface extends another interface**.

```
interface Printable{
    void print();
}

interface Showable extends Printable{
    void show();
}

class TestInterface2 implements Showable {
    public void print(){
        System.out.println("Hello");
    }
    public void show(){
        System.out.println("Welcome");
    }

    public static void main(String args[]){
        TestInterface2 obj = new TestInterface2();
        obj.print();
        obj.show();
    }
```



APPLICATION

Create a class hierarchy to represent different types of vehicles (e.g., cars, trucks, motorcycles) with their specific properties and methods. Based on the following:

a. Base class is vehicle with
   - data fields/members: color and number of wheels
   - abstract methods: start() and stop().

b. Derived classes:
   - Car {}
   - Truck {}
   - Motorcycle {}

Nice work, students! The lesson is complete, keep going!

Inheritance

WEEK 11 TO WEEK 12

LEARNING OUTCOMES

1. Explain the concept of inheritance and analyze how subclasses acquire properties and behaviors from superclasses in Java.
2. Recognize the value of inheritance in reducing redundancy, and express willingness to reuse code through superclass design in programming.
3. Implement inheritance in Java by creating subclasses that inherit from a superclass.

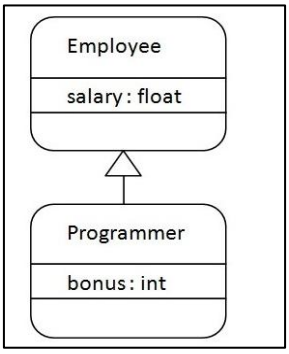
INTRODUCTION

Welcome to Module 3, Lesson 2, dear students! In this lesson, you will learn how to create a hierarchical structure of classes and reuse code efficiently. Let us dive in and see how inheritance can help you solve real-world programming problems.



ACTIVITY

Examine the figure carefully to answer the questions.





ANALYSIS

1. How are these two classes connected?
2. Which class is extended by the other?
3. Can you determine the superclass and the subclass? If yes, please elaborate how you arrived at that conclusion.

Inheritance in Java is a mechanism in which one class/object acquires all the properties *(attributes)* and behaviors *(methods)* of another class/object. Inheritance means you can create new classes based on existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can also add new methods and fields to your current class. Inheritance represents the *IS-A relation*ship, which is also known as a *parent-child relationship*.

Term Used In Inheritance

Class is a group of objects which have common properties.

Subclass, also known as a child class, is a class which inherits from another class. It is also referred to as a derived class, an extended class, or a child class.

Superclass/Parent Class is the class from where a subclass inherits the features. It is also referred to as a base class or a parent class.

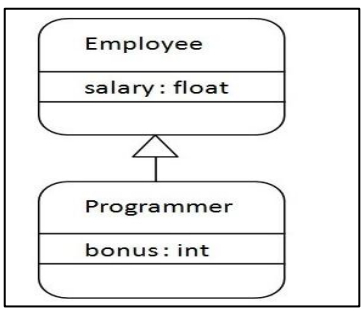
Reusability is a mechanism that facilitates the reuse of fields and methods from an existing class when creating a new class. You can use the same fields and methods already defined in the previous class.

The ***extends*** keyword indicates that you are making a new class that derives from an existing class. The meaning of 'extends' is to increase the functionality.

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

A class that is inherited is referred to as a *parent or superclass*, while the new class is called a *child or subclass*.

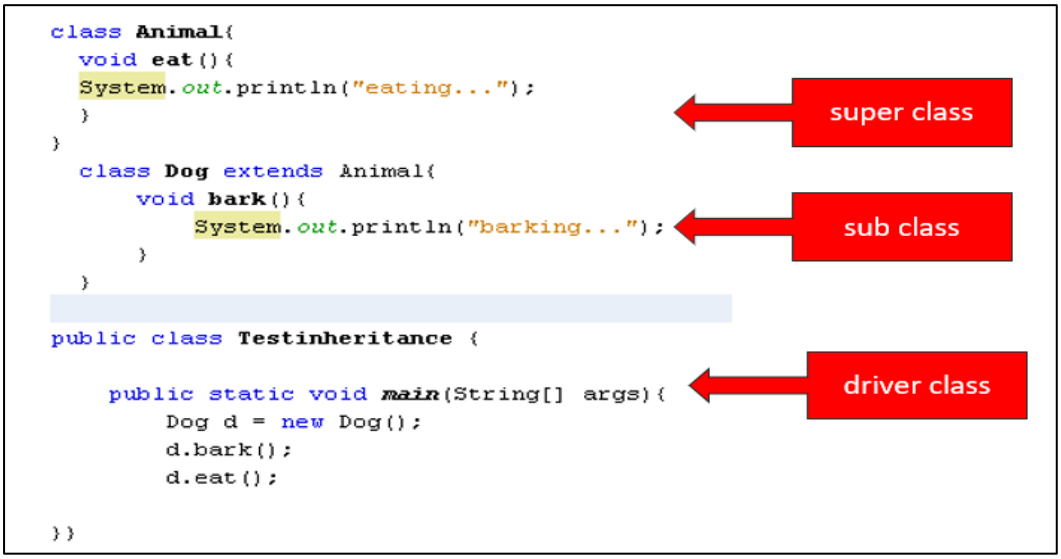
Java Inheritance Example



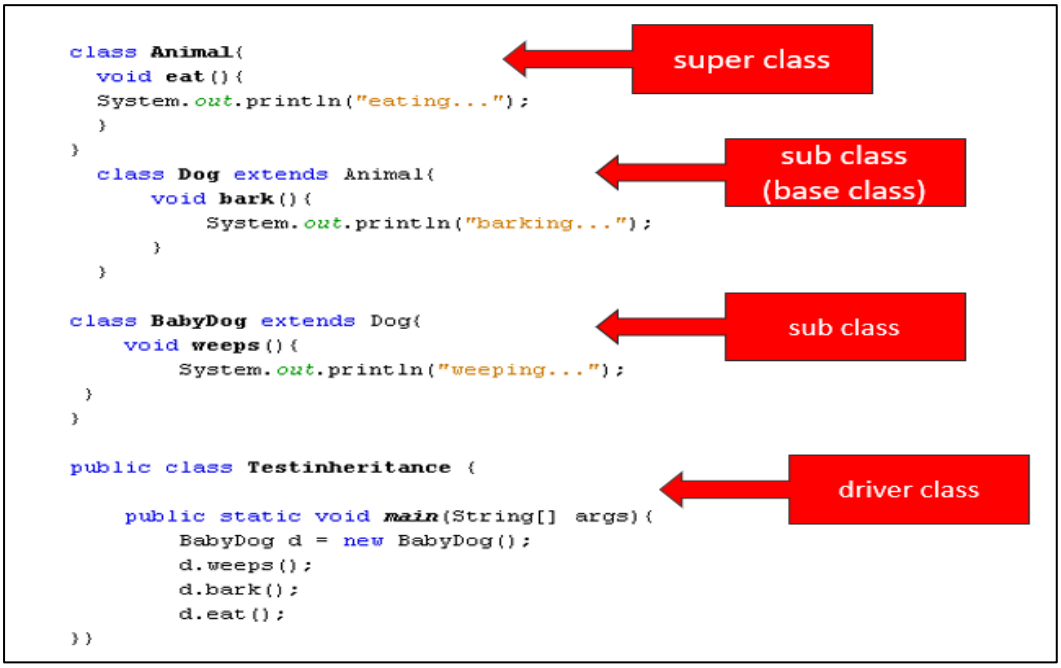
As shown in the above figure, *Programmer* is a subclass and *Employee* is its superclass. The relationship between the two classes is *Programmer* IS-A *Employee*. It means that a *Programmer* is a type of *Employee*.

Types Of Inheritance

Single Inheritance: a *subclass* is derived from only one *superclass*. It inherits the properties and behavior of a single-parent class. Also known as *simple inheritance*.



Multilevel Inheritance: a *derived class* will be inheriting a *base class*, and the *derived class* also acts as the *base class* for other classes alternatively, when there is a chain of inheritance.



Hierarchical Inheritance: one class serves as a *superclass* for more than one *subclass*, when two or more classes inherit a single class.

```
class Animal{
  void eat(){
    System.out.println("eating...");
  }
}
  class Dog extends Animal{
      void bark(){
          System.out.println("barking...");
      }
  }

class cat extends Animal{
    void meows(){
        System.out.println("meowing...");
 }
}

public class Testinheritance {

    public static void main(String[] args){
        cat c = new cat();
        c.meows();
        c.eat();

}}
```


Multiple Inheritance : one class can have more than one *superclass* and inherit features from all parent classes. However, Java does not support multiple inheritances with classes. We can achieve multiple inheritance only through *Interfaces*.

```
interface One{
    public void print_geek();
}

interface Two{
    public void print_for();
}

interface Three extends One,Two{
    public void print_geek();
}

class Child implements Three{
    public void print_geek(){
        System.out.println("Geeks");
    }
    public void print_for(){
        System.out.println("for");
    }
}
public class MultipleInheritanceInterface {

   public static void main(String args[]){
       Child c = new Child();
        c.print_geek();
        c.print_for();
        c.print_geek();

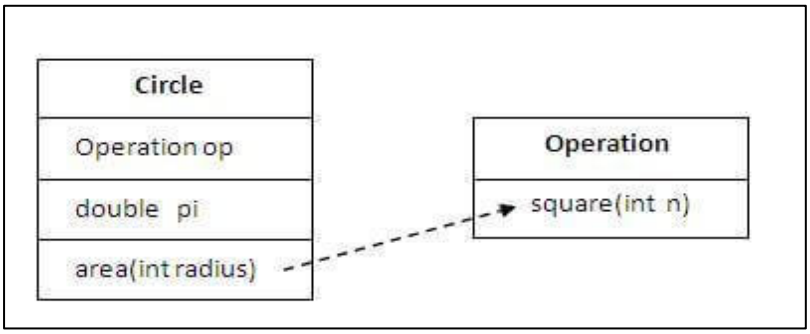
   }
}
```

The ***final*** keyword is used to prevent some classes from being inherited or extended.

```
final class Vehicle {
  ...
}

class Car extends Vehicle {
  ...
}
```

Aggregation represents a *HAS-A* relationship. Code reuse is also best achieved through Aggregation.




```
public class EMp {
    int id;
    String name;
    Address address;

    public EMp(int id, String name,Address address){
        this.id=id;
        this.name=name;
        this.address=address;
    }

    void display(){
      System.out.println(id + " " +name);
      System.out.println(address.municipality+ " " + address.province);

}

    public static void main(String[] args){
        Address address1 = new Address("Hagonoy","Davao del Sur");
        Address address2 = new Address("Digos","Davao del Sur");

        EMp e=new EMp(2,"razel",address1);
        EMp e2=new EMp(6,"vern",address2);

        e.display();
        e2.display();
    }
}
```

Advantages of Inheritance

Code Reusability: Code written in the *superclass or parent class* is common to all *subclasses or child classes*.

Method Overriding: Achievable only through inheritance and one of the ways that Java achieves run-time polymorphism.

Abstraction: Achieved through inheritance. Abstraction only shows the functionality to the user.

Class Hierarchy: Used to model real-world objects and their relationships.

Polymorphism: Inheritance enables polymorphism. *Subclasses* can override the methods of their *superclass,* allowing them to change their behavior in different ways.

Disadvantages of Inheritance

Complexity: The code becomes more complex and challenging to understand when multiple inheritance is used.

Tight Coupling: Creates a tight coupling between the *superclass* and *subclass*, making it difficult to make changes to the *superclass* without affecting the *subclass*.



APPLICATION

Consider the following code, what will be the output of the code?

```
class Parent {
    int x = 10;
 }
class Child extends Parent {
   int y = 20;
  void display() {
   System.out.println("x = " + x + ", y = " + y);
  }
}
 public class SuperKeywordExample {
public static void main(String[] args) {
 Child child = new Child();
 child.display(); }
}
```

Great job, students! You're all set, keep the momentum going and get ready for what's next!.

Module 3 Lesson 3

Polymorphism

WEEK 13 TO WEEK 14

LEARNING OUTCOMES

1. Explain the concept of polymorphism and distinguish between method overloading and method overriding in Java.
2. Analyze how polymorphism enhances flexibility and scalability in software development.
3. Implement polymorphism in Java programs using overriding and overloading methods.

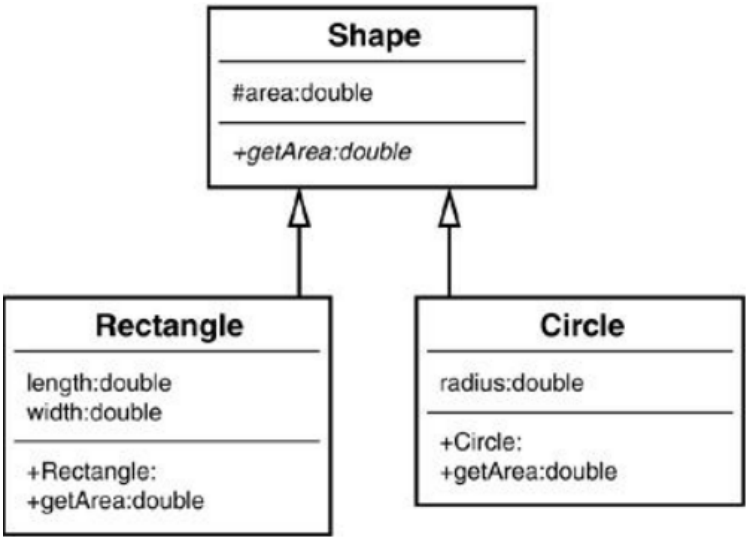
INTRODUCTION

Ready to unlock the potential of polymorphism, dear students? Get ready for a deep dive into this fundamental OOP concept and its practical applications.



ACTIVITY

Take a moment to examine the figure and see if you can answer the questions based on what you observe.





ANALYSIS

1. Which class is the base class, and which classes are derived from it?
2. What do you think is the overall concept being demonstrated in this figure?

Polymorphism is a concept that allows to perform a single action in different ways. Derived from two Greek words, "poly" means "many" and "morphs" means "forms". So, polymorphism means many forms. It occurs when there is inheritance and it enable us to utilize these inherited properties to perform various tasks.

Types of Polymorphism

Polymorphism in two different methods:

- **Method Overloading** is the process that can create multiple methods of the same name in the same class, and all the methods work in different ways. It occurs when a class has more than one method with the same name.
    - *Compile-Time Polymorphism (static)*
- **Method Overriding is the process by which a subclass or child class declares a method with the same name as that of the parent class.**
    - *Run-Time Polymorphism (dynamic)*

Method Overloading

```
class shapess{
    public void area(){
        System.out.println("Find area");
}

    public void area(int r){
        System.out.println("Circle area= " + 3.14*r*r);
    }
    public void area(double b, double h){
        System.out.println("Triangle area= " +0.5*b*h);
    }
    public void area(int l, int b){
        System.out.println("Rectangle area= " +l*b);
    }
}

class Shape2 {

public static void main(String[] args){
    shapess myshapess = new shapess();
    myshapess.area();
    myshapess.area(5);
    myshapess.area(6.0, 2.0);
    myshapess.area(6, 2);
    }
}
```

Method overloading increases the readability of the program. There are two ways to overload the method in Java

By changing number of arguments. In this example, we have created two methods: first *add() method* performs the addition of two numbers and second *add() method* performs addition of three numbers. We are creating *static* methods so that we do not need to create instance for calling methods.

```
public class Adder {

     static int add(int a, int b){
          return a+b;
     }
     static int add(int a, int b, int c){
          return a+b+c;
     }

}

class TestOverloading1{
     public static void main(String[] args){
          System.out.println(Adder.add(11, 11));
          System.out.println(Adder.add(11, 11, 11));
}}
```

By changing the data type: In this example, we have created two methods that differs in: first *add() method* receives *two integer arguments*. Second *add() method* receives *two double arguments*.

```
public class Adder {

     static int add(int a, int b){
          return a+b;
     }
     static double add(double a, double b){
          return a+b;
     }

}

class TestOverloading1{
     public static void main(String[] args){
          System.out.println(Adder.add(11, 11));
          System.out.println(Adder.add(12.3, 12.6));
}}
```

Overloading a method is not possible by changing the return type of the method only due to ***ambiguity***.

```
class Adder{
static int add(int a,int b){return a+b;}
static double add(int a,int b){return a+b;}
}
class TestOverloading3{
public static void main(String[] args){
System.out.println(Adder.add(11,11));//ambiguity
}}
```

Can we overload the Java main() method? Yes, you can have any number of main methods in a class by overloading. However, the JVM calls the *main() method, which receives a string array as arguments only.*

```
class TestOverloading{
public static void main(String[] args){
System.out.println("main with String[]");
  }
public static void main(String args){
System.out.println("main with String");
  }
public static void main(){
System.out.println("main without args");
  }
}
```

Method Overriding

When a ***subclass*** has the same method as declared in the parent class, an overriding method is used to provide a specific implementation of a ***method*** that is already provided by its ***superclass***. Method overriding is used for ***runtime polymorphism***.

Rules for method overriding:

1. The method must have the same name as in the parent class.
2. The method must have the same parameters as in the parent class.
3. There must be an IS-A relationship *(inheritance)*.

```java
public class Vehicle {
     void runs(){
          System.out.println("Vehicle is running");}
    }

class Bikee extends Vehicle{
     void runs(){
          System.out.println("Bike is running safely");
     }

     public static void main(String[] args ){
          Bikee objb=new Bikee();
          objb.runs();

     }
}
```

Method Overriding

```java
//creating parent class
 class Bank {
 int getRateOfInterest(){
     return 0;}
}
//creating child classes
class SBI extends Bank{
    int getRateOfInterest(){
        return 8;}
}
class ICICI extends Bank{
    int getRateOfInterest(){
        return 7;}
}
class AXIS extends Bank{
    int getRateOfInterest(){
        return 9;}
}

//test class to create objects and call the methods
class Test2{
    public static void main(String[] args){
        SBI s=new SBI();
        ICICI i=new ICICI();
        AXIS a=new AXIS();

        System.out.println("SBI Rate of Interest: " +s.getRateOfInterest());
        System.out.println("SBI Rate of Interest: " +i.getRateOfInterest());
        System.out.println("SBI Rate of Interest: " +a.getRateOfInterest());
    }
```

*The **Java main() method cannot be overridden because it is a static method**. A static method cannot be overridden* because it is bound to the class, whereas an instance method is bound to an object. Overriding is done by using a reference variable of the *superclass*. The *method* to be called is determined based on the object referred to by the reference variable. If the *reference variable* of a *parent class* refers to an object of a *child class*, it is known as ***upcasting***.

Runtime Polymorphism

Runtime polymorphism (Dynamic Method Dispatch) is a process in which a call to an overridden method is *resolved at runtime rather than compile time*. In this process, an overridden method is called through the reference variable of a *superclass*. Method overriding occurs when a *subclass* defines a *method with the same name, parameters, and return type as those of the superclass*. That method overrides the function in the superclass.

In simpler terms, if the *subclass* provides its definition to a *method* already present in the *superclass*; then that *method* in the *superclass* is said to be *overridden*. Additionally, *runtime polymorphism* can only be achieved through *methods*, not data members.

```
public class Bike {

    void run(){
    System.out.println("running");
    }
}

    class Splendor extends Bike{
    void run(){
        System.out.println("running safely with 60km");

    }
        public static void main(String[] args){
            Bike b = new Splendor();//upcasting
            b.run();
        }
   }
```

Example: Superclass is *Shape,* classes *Rectangle, Circle and Triangle* extends *Shape* class and overrides its *draw() method.*

```
public class Shape {

    void draw(){
        System.out.println("drawing");}
}
class Rectangle extends Shape{
    void draw(){
        System.out.println("drawing rectangle");}
}

class Circle extends Shape{
 void draw(){
     System.out.println("drawing circle");}
}

class Triangle extends Shape{
    void draw(){
       System.out.println("drawing triangle");}
}

class TestPolymorphism{
    public static void main(String[] args){
        Shape s;
        s= new Rectangle();
        s.draw();
        s=new Circle();
        s.draw();
        s=new Triangle();
        s.draw();
        }
```

Example: Created a class *animal, dog, cat and lion* with polymorphism with a *sound() method*.

```
public class Animal {
        void sound(){
         System.out.println("create a sound"
    }

class Dog extends Animal{
          void sound(){
         System.out.println("bark");}
    }
class Cat extends Animal{
          void sound(){
         System.out.println("Meow");}
    }
class Lion extends Animal{

     void sound(){
         System.out.println("Roar");}
    }

class Polymorphism{
public static void main(String[] args){
    Animal a;
    a=new Dog();
    a.sound();
    a=new Cat();
    a.sound();
    a=new Lion();
    a.sound();
    }
}
```

Runtime Polymorphism with multilevel inheritance

Example: One superclass *Animal* and three subclasses, H*erbivores, Carnivores, and Omnivores*. Subclasses extend the superclass and override its eat() method.

```
public class Animal3 {
    void eat(){
        System.out.println("animal is eating");}
    }

class dinosaur extends Animal3{
    void eat(){
        System.out.println("dinosaur is eating");}
    }

class BabyDinosaur extends dinosaur{

    public static void main(String[] args){
        Animal3 a3;
        a3=new BabyDinosaur();
        a3.eat();
    }
}
```

```
class Animal12{
void eat(){
    System.out.println("Animals eat");}
}

class herbivores extends Animal12{
    void eat(){
        System.out.println("Herbivores eat plants");}
}

class omnivores extends Animal12{
    void eat(){
        System.out.println("Omnivores eat plants and meat");}
}

class carnivores extends Animal12{
    void eat(){
        System.out.println("Carnivores eat meat");}
}

public class Animal1 {
    public static void main(String[] args){
        Animal12 A = new Animal12();
        Animal12 H = new herbivores(); //upcasting
        Animal12 O = new omnivores(); //upcasting
        Animal12 C = new carnivores();//upcasting
        A.eat();
        H.eat();
        O.eat();
        C.eat();
    }
```

Compile-time Polymorphism

Compile-time polymorphism (static polymorphism) can be achieved through *method overloading*. This enables us to perform various operations by using *multiple methods having the same name*.

```
public class Addition {

    void sum(int a, int b){
        int c=a+b;
        System.out.println("Addition of two numbers: "+c);
      }
    void sum(int a,int b,int e){
        int c=a+b+e;
        System.out.println("Addition of three numbers: " +c);
    }

    public static void main(String[] args){
        Addition a=new Addition();
        a.sum(30, 90);
        a.sum(45, 80, 22);
    }

}
```



APPLICATION

Create a Java program that demonstrates method overloading by displaying the locations and the names of colleges or universities associated with them. Given that the base class is Location, with methods displayLocation() and displayStateCollege().

Sample Output:

```
location: Digos City
SUC: Davao del Sur State College
location:Panabo
SUC: Davao del Norte State College
location: North Cotabato
SUC: University of Southern Mindanao
```

Nice work, students! Keep moving forward, you're doing great!

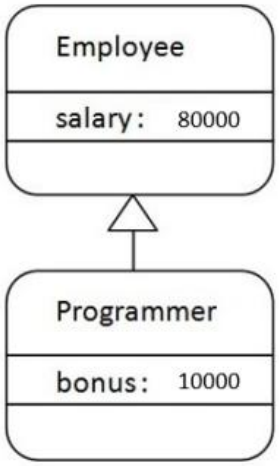
MODULE ASSESSMENT

1. Create a Java program that demonstrates polymorphism by calculating the areas of different shapes, namely rectangles and triangles. Implement a base Shape class with a calculateArea() method, and then create derived classes for each shape, overriding the calculateArea() method with the appropriate formula.

Formula:

- A=bh (area of rectangle, where b is the base, h is the height)
- A=1/2 bh (area of triangle)

2. Given the figure below. Write a program code that shows their relationship.



Sample Output:

```
run:
Salary: 80000
Benefits :10000
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Use Aggregation in a program where you will display the name, year, program, and department of three students.

Sample Output:

```
Jon, 2, BSIT, Computing Department
Arya 1, BSIS, Computing Department
Bran 4, BSIT, Computing Department
```

MODULE SUMMARY

Abstraction focuses on simplifying complex systems by hiding internal details and presenting only essential functionalities. This is achieved through abstract classes, which can have abstract methods and provide a framework for subclasses. Interfaces further enhance abstraction by defining a contract of methods that classes must implement, promoting loose coupling and enabling multiple inheritance.

Inheritance enables code reusability by allowing new classes to inherit properties and behaviors from existing classes. It establishes a "parent-child" relationship and supports various inheritance types, including single, multilevel, and hierarchical.

Polymorphism, derived from the ability to take on multiple forms, allows objects of different classes to be treated as objects of a common type. This is achieved through method overloading (using the same method name with different parameters) and method overriding (where a subclass provides its implementation of a superclass method), enabling flexibility and dynamic behavior in Java programs

REFERENCES

Burd, B. (2021). *Beginning programming with java for dummies*. John Wiley & Sons, Inc.

Budd, T.A. (2021). *An Introduction to object-oriented programming*. Addison Wesley.

Sarcar, V. (2020). *Interactive object-oriented programming in java*. Springer Science.

3G E-Learning. (2023). *Object-oriented Programming*.

MODULE 04:
File Handling and Database Connectivity

Welcome to the last module, dear students! You've made it this far, great job! In module 4, you'll explore the core principles and methods for working with files and databases in the Java programming language. These critical components of software development enable applications to interact with external data sources, store and retrieve information, and preserve data beyond the application's runtime.

This module is divided into 2 lessons:

Lesson 1: File Input/Output Streams
Lesson 2: Database Connectivity

At the completion of this module, you should be able to:

1. Design, implement, test, and debug programs using OOP concepts such as abstraction, encapsulation, inheritance, and polymorphism.

Take your time, practice the examples, and enjoy applying your skills to meaningful programming tasks. You're now at the final stretch of your OOP journey!

Module 4 Lesson 1

File Input/Output Streams

WEEK 15

LEARNING OUTCOMES

1. Explain the purpose and types of file input/output streams in Java and differentiate between byte and character streams.
2. Demonstrate an understanding of the importance of file handling in building real-world applications.
3. Write Java programs that use file input/output streams to read from and write to files using appropriate classes and exception handling.

INTRODUCTION

It is time to dive into Lesson 1 of Module 4, dear students. This lesson focuses on file streams. We will learn how to perform basic operations, such as creating, deleting, reading, and writing data.



ACTIVITY

Create a file named input.txt containing "Hello World". Then, implement the following code and analyze its output to answer the questions below.

```
public static void main(String args[]) throws IOException {
   FileInputStream in = null;
   FileOutputStream out = null;

   try {
      in = new FileInputStream("input.txt");
      out = new FileOutputStream("output.txt");

      int c;
      while ((c = in.read()) != -1) {
         out.write(c);
      }
   }finally {
      if (in != null) {
         in.close();
      }
      if (out != null) {
         out.close();
      }
   }
}
```



ANALYSIS

1. What specific classes are employed to perform file I/O operations in the given code?

2. What happens when you compile and execute the program?



ABSTRACTION

File Handling is an integral part of any programming language, as it enables us to store the output of a program in a file and perform specific operations on it.

In simple terms, file handling refers to reading and writing data to a file. In Java, the concept of a stream is used to perform I/O operations on a file. Stream can be defined as a sequence of data.

There are two kinds of Streams:

- *InPutStream* - read data from a source.
- *OutPutStream* - writing data to a destination.

File Operations in Java

Several operations that can be performed on a file in Java : *Create a File, Read from a File, Write to a File and Delete a File.*

Create a File: Use the *createNewFile() method*. If the file is successfully created, it will return a Boolean value true; otherwise, it will return false if the file already exists.

```
import java.io.File; //import the file class
import java.io.IOException;

public class CreateaFile {
    public static void main(String[] args)
    {

        try {
            File Obj = new File("myfile.txt");
            if (Obj.createNewFile()) {
                System.out.println("File created: " + Obj.getName());
            }
            else {
                System.out.println("File already exists.");
            }
        }
        catch (IOException e) {
            System.out.println("An error has occurred.");
            e.printStackTrace();
        }
    }
}
```

Read From a File : Use the Scanner class

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadFromFile {

    public static void main(String args[]){
        try {
            File Obj = new File("myfile.txt");
            Scanner Reader = new Scanner(Obj);
            while (Reader.hasNextLine()) {
                String data = Reader.nextLine();
                System.out.println(data);
            }
            Reader.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("An error has occurred.");
            e.printStackTrace();
        }
    }

}
```

Write a File: Use the *FileWriter class*, along with its *write() method* file.

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteToAFile {
    public static void main(String args[]){
        try {
            FileWriter Writer  = new FileWriter("myfile.txt");
            Writer.write("Files in Java are seriously good!!");
            Writer.close();
            System.out.println("Successfully written.");
        }
        catch (IOException e) {
            System.out.println("An error has occurred.");
            e.printStackTrace();
        }
    }
}
```

Delete a File : Use the *delete() method*

```
import java.io.File;

public class DeleteAFile {

    public static void main(String args[]){
         {
        File Obj = new File("myfile.txt");
        if (Obj.delete()) {
            System.out.println("The deleted file is : " + Obj.getName());
        }
        else {
            System.out.println( "Failed in deleting the file.");
        }
      }
    }
}
```

Byte Streams are used to perform input and output *of 8-bit bytes : FileInputStream and FileOutputStream.*

```
 public static void main(String args[]) throws IOException {
   FileInputStream in = null;
   FileOutputStream out = null;

   try {
      in = new FileInputStream("input.txt");
      out = new FileOutputStream("output.txt");

      int c;
      while ((c = in.read()) != -1) {
         out.write(c);
      }
   }finally {
      if (in != null) {
         in.close();
      }
      if (out != null) {
         out.close();
      }
   }
}
```

Character Streams are used to perform input and output of *16-bit bytes*: *FileReader and FileWriter* (which read and write two bytes at a time).

```
    public static void main(String args[]) throws IOException {
      FileReader in = null;
      FileWriter out = null;

      try {
         in = new FileReader("input.txt");
         out = new FileWriter("output.txt");

         int c;
         while ((c = in.read()) != -1) {
            out.write(c);
         }
      }finally {
         if (in != null) {
            in.close();
         }
         if (out != null) {
            out.close();
         }
      }
   }
```

Example 1: Read standard input until the user types a character 'q'.

```
public static void main(String args[]) throws IOException {
   InputStreamReader cin = null;

   try {
      cin = new InputStreamReader(System.in);
      System.out.println("Enter characters, 'q' to quit.");
      char c;
      do {
         c = (char) cin.read();
         System.out.print(c);
      } while(c != 'q');
   }finally {
      if (cin != null) {
         cin.close();
      }
   }
}
```

Example 2: Read standard input until the user types "quit". Instead of reading characters one by one, we read a whole line of input using BufferedReader.readLine()

```java
public static void main(String args[]) throws IOException {
  InputStreamReader cin = null;

  try {
    cin = new InputStreamReader(System.in);
    System.out.println("Enter a message (or 'quit' to exit): ");

    String input;
    do {
      // Read a line of input
      input = new BufferedReader(cin).readLine();
      if (input != null) {

      }
    } while (!input.equalsIgnoreCase("quit"));     // Check for "quit" (case-insensitive)

    System.out.println("Exiting program...");
  } finally {
    if (cin != null) {
      cin.close();
    }
  }
 }
}
```

Example 3: Program that reads and writes to text file and terminates with the String "end" (ignoring case)

```java
  public static void main(String args[]) throws IOException {
    BufferedReader reader = new BufferedReader(new FileReader("in.txt"));
      BufferedWriter writer = new BufferedWriter(new FileWriter("ou.txt"));

      String line;
      while((line = reader.readLine()) != null)
      {
          if (line.equalsIgnoreCase("end")) {
              break;
          }
          writer.write(line);
          writer.newLine();
      }

      reader.close();
      writer.close();
  }
```

Example 4: Adding two numbers (input in different lines)

```
//create input/output streams
BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
 BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"));

 //read numbers from the input file (in different lines)
String line1=reader.readLine();
String line2=reader.readLine();

//parsing/ parse the numbers (convert string to int)
int num1=Integer.parseInt(line1);
int num2=Integer.parseInt(line2);

//calculate
int sum=num1+num2;

//write the result to output file
writer.write("Sum: = " + sum);

//close the streams
reader.close();
writer.close();
```

Example 5: Adding two numbers (input in a single line)

```
//create input/output streams
BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
 BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"));

 //read the line containing numbers
String line=reader.readLine();

//tokenize the line to separate the numbers
StringTokenizer tokenizer = new StringTokenizer(line);

//extract the numbers
int num1=Integer.parseInt(tokenizer.nextToken());
int num2=Integer.parseInt(tokenizer.nextToken());

//calculate
int sum=num1+num2;

//write the result
writer.write("Sum: = " + sum);

//close the streams
reader.close();
writer.close();
```

APPLICATION

Create a program to calculate the Sale price of an item with a 20% discount. Input should be read from the text file "pricein.txt" in a single line and written to the text file "priceout.txt".

Sample input:

File Edit View

```
1999.75 0.20
```

Sample output:

File Edit View

```
Original Price:  1999.75
Sale Price:  1599.8
```

Great effort, students ! You've made strong progress, keep moving forward with confidence as you take on the last topic!

Module 4 Lesson 2

Database Connectivity

WEEK 16 TO WEEK 17

LEARNING OUTCOMES

1. Explain the concept of database connectivity and describe how Java Database Connectivity (JDBC) is used to connect Java applications with relational databases
2. Show appreciation for the role of database connectivity in building dynamic applications.
3. Design a simple Java program with a graphical User Interface (GUI) that that connects to a database, retrieves, and displays it using JDBC.

INTRODUCTION

Dear students, welcome to Module 4, Lesson 2, the final topic of this course! This lesson introduces you to the JDBC API and its importance in connecting Java applications to databases. By the end of this lesson, you will build a solid foundation in database concepts and learn how to use Java to interact with databases effectively.



ACTIVITY

Assume that you are tasked with creating a simple student management application using Java and a relational database. The application should allow users to add, edit, delete, and view student records.



ANALYSIS

How would you design the database schema for the student management application? What tables would you create, and what columns would they contain?



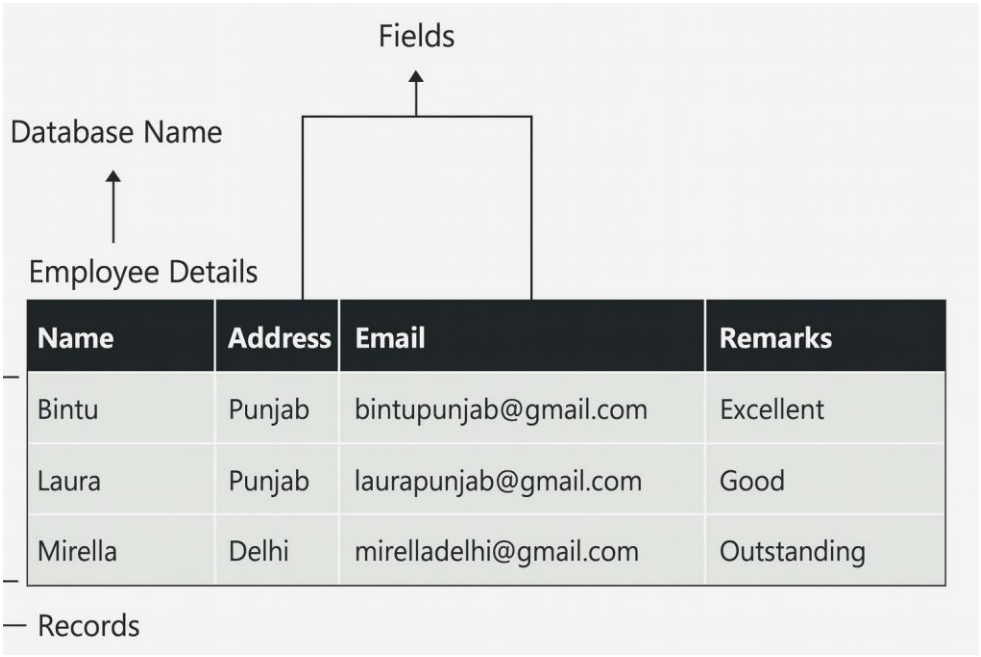
ABSTRACTION

Applications commonly employ *Forms* to collect user input. These Forms constitute the *User Interface (UI),* where users enter data via a keyboard. Java provides *the AWT* **and** *Swing libraries* as frameworks for designing and implementing the UI for various applications. There are two ways for storing the data in the computer: *files and database*

Overview Of Java Database

Database is a collection of large amounts of related data in the form of *tables*.

Tables are a collection of *rows and columns*. The data contained in a table is in form of fields such as name, address, phone, email and remark, which are also called *columns*.



JDBC is a set of Java Application Programming Interfaces (APIs) that enables you to access any tabular data source using the Java programming language.

Characteristics of JDBC

1. Provides Call level interface (CLI) that enables Java to interact with database. CLI enables you to use SQL statements.
2. Provides the DBMS driver that passes the SQL query to enable connectivity.
3. Provides a simple mechanism to access data from a RDBMS.

Advantages of JDBC

1. Enables you to manipulate data on enterprise level.
2. Does not require any additional software, as Java has built-in software for JDBC.
3. Dose not need any configuration on the client side for connectivity.

JDBC API: Provides a standard set of interfaces and classes that enable Java developers to write applications that interact with various databases. JDBC

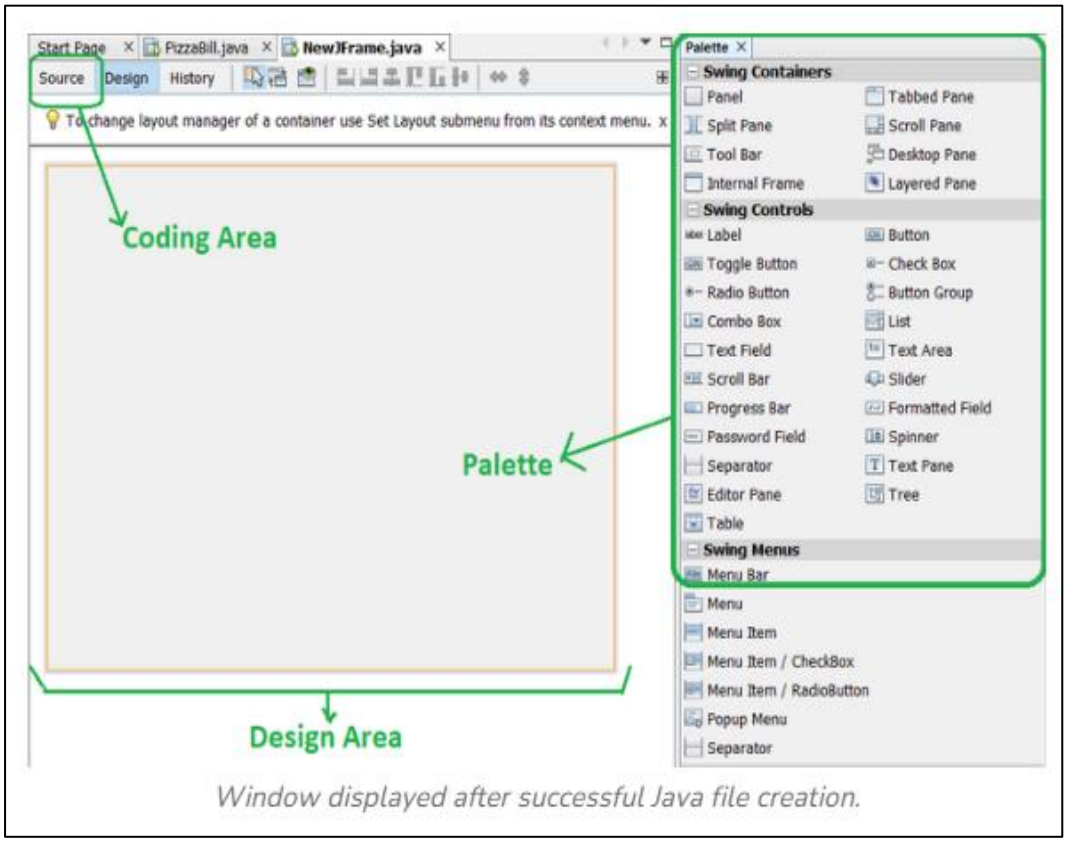
acts as an intermediary, facilitating communication between the Java application and the database system

JDBC Driver: To connect to different databases, you must utilize specific JDBC drivers designed for each respective database type."

Creating an application

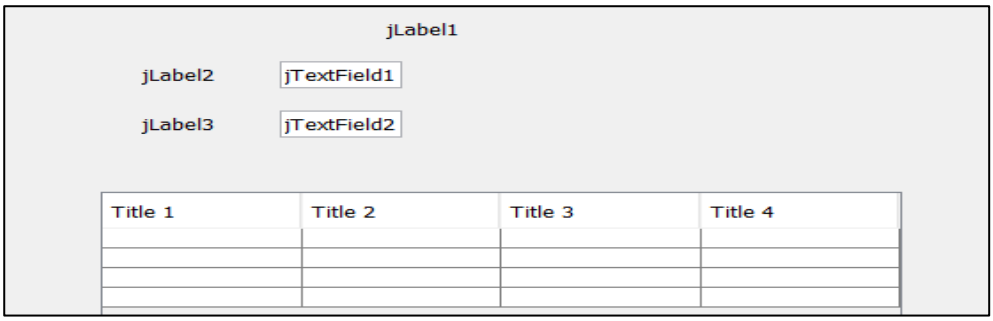
Java JFrame Form

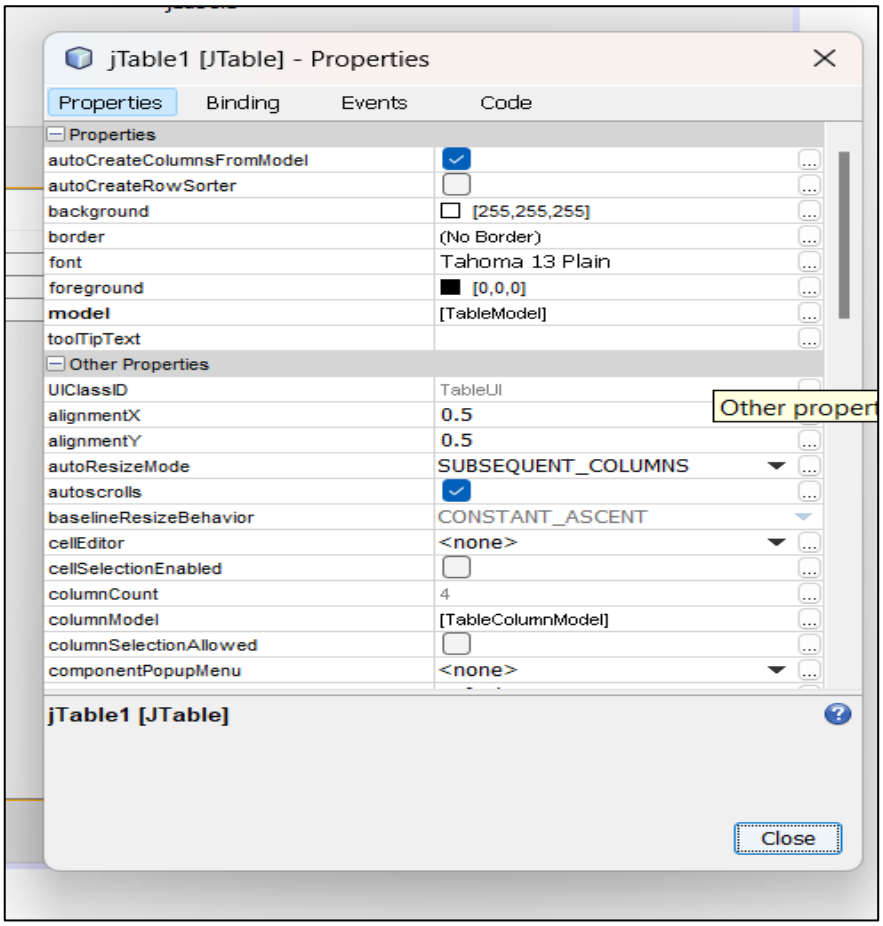
- Design: create the design/template.
- Source: logic code of the program is written.
- Palette: Provides a selection of widgets that can be dragged and dropped onto the design area



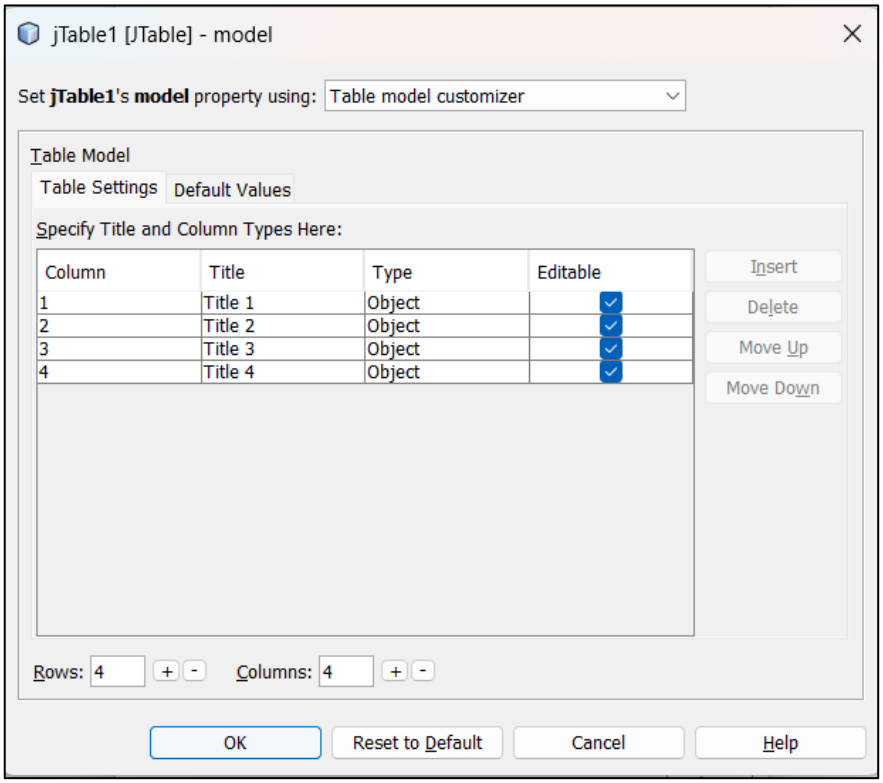
Window displayed after successful Java file creation.

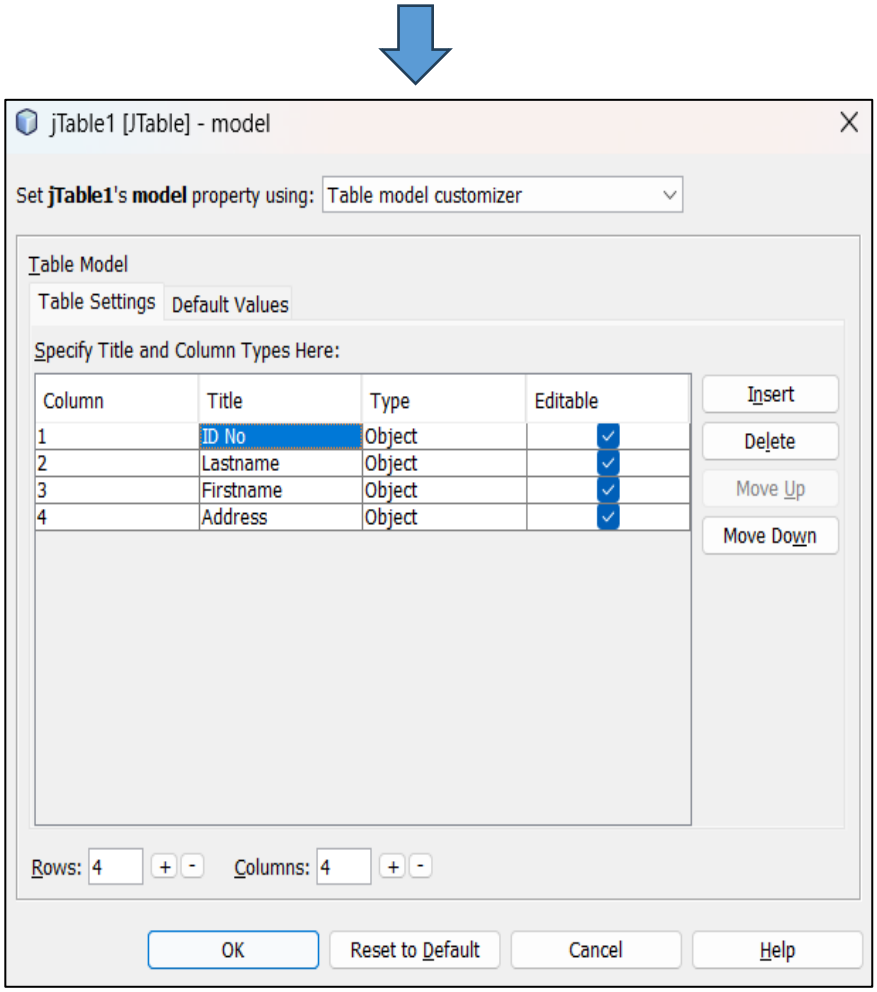
JTable: Right click > properties > Model > add/delete

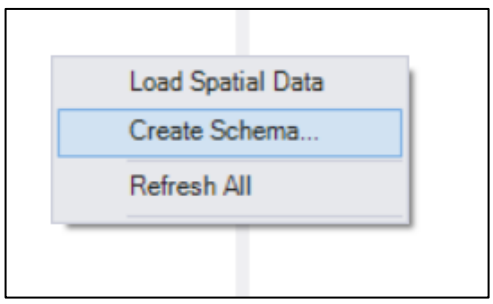




Creating the Database and Table: Create database (Schema) named *Student*




jTable1 [JTable] - model
Set jTable1's model property using:
Table model customizer
Table Model
Table Settings
Default Values
Specify Title and Column Types Here:
Column
Title
Type
Editable
1
ID No
Object
2
Lastname
Object
3
Firstname
Object
4
Address
Object
Insert
Delete
Move Up
Move Down
Rows: 4
Columns: 4
OK
Reset to Default
Cancel
Help


Load Spatial Data
Create Schema...
Refresh All

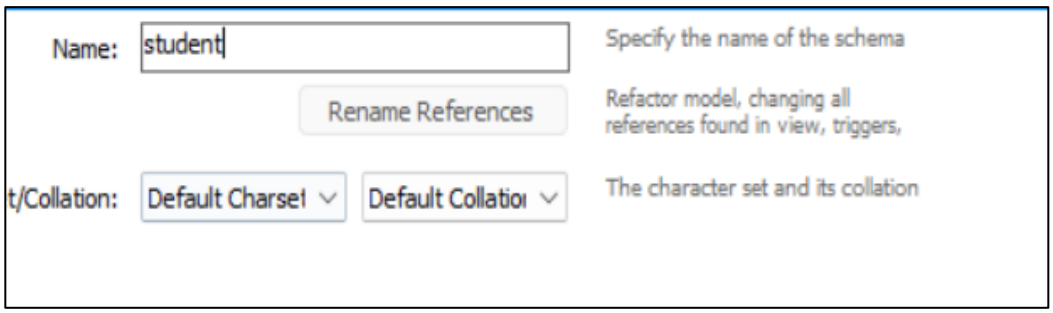
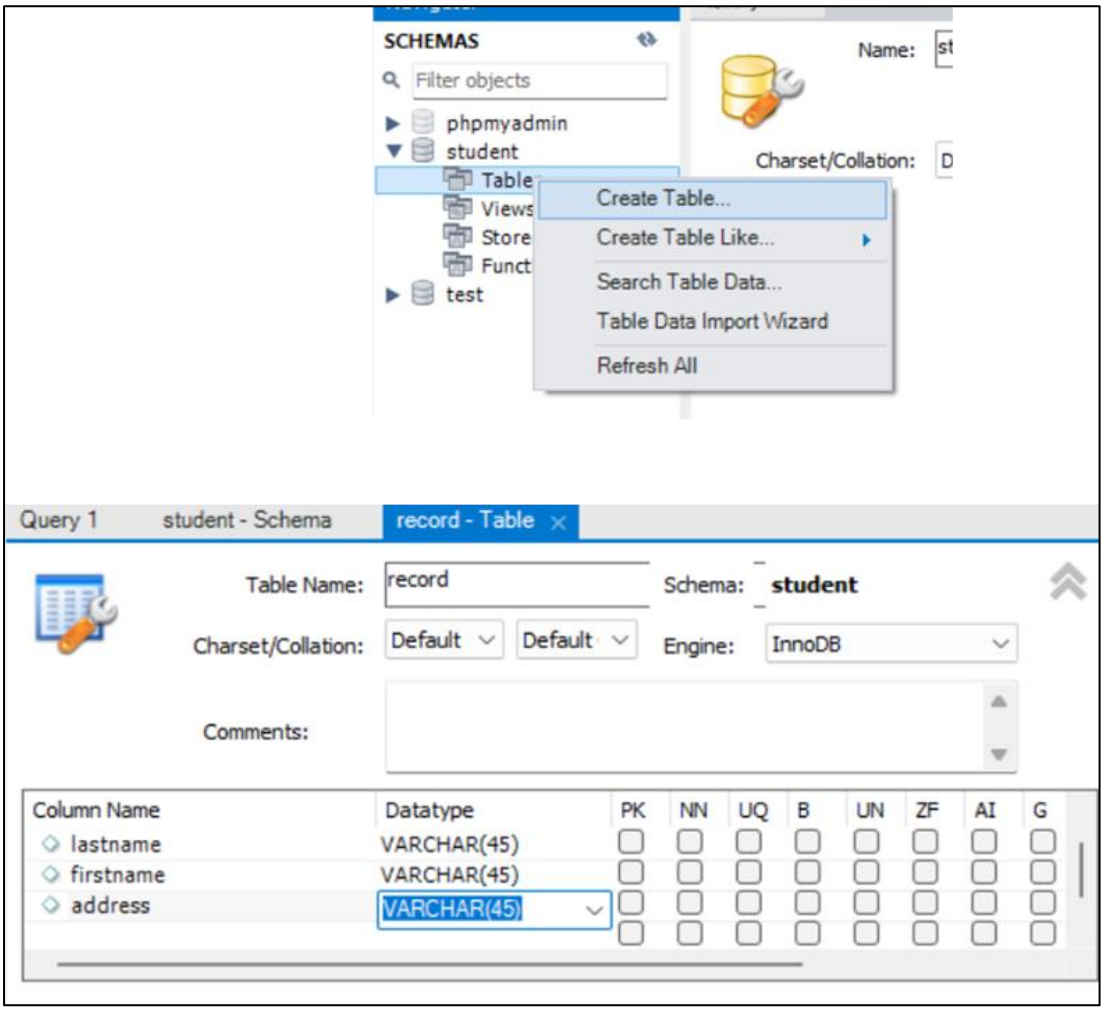
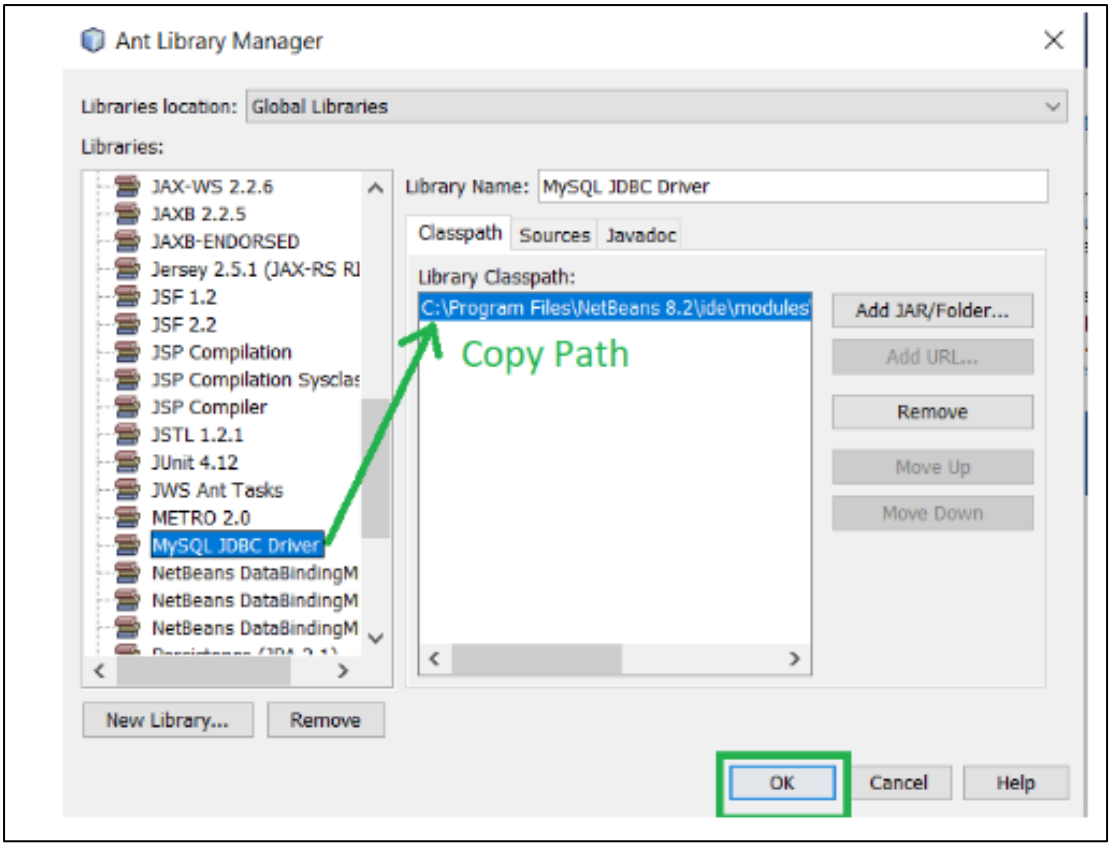

Name:
student
Specify the name of the schema
Rename References
Refactor model, changing all references found in view, triggers,
t/Collation:
Default Charset
Default Collatio
The character set and its collation

Table: Record

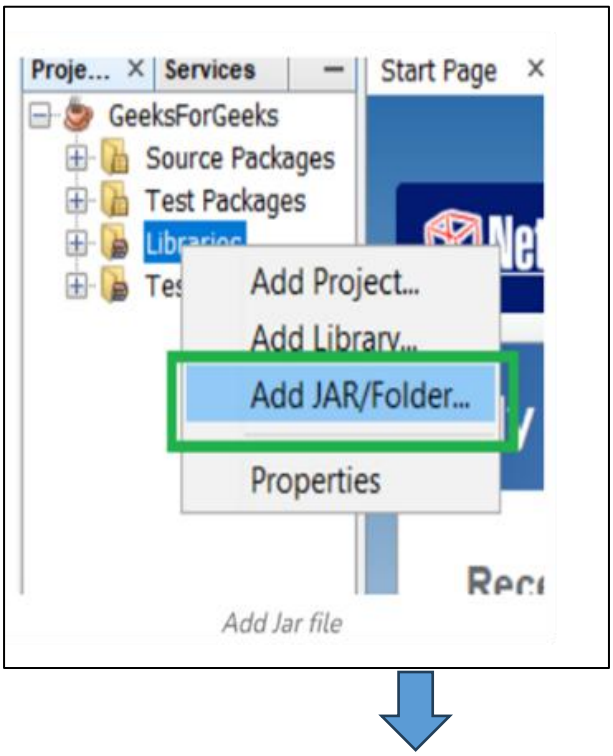


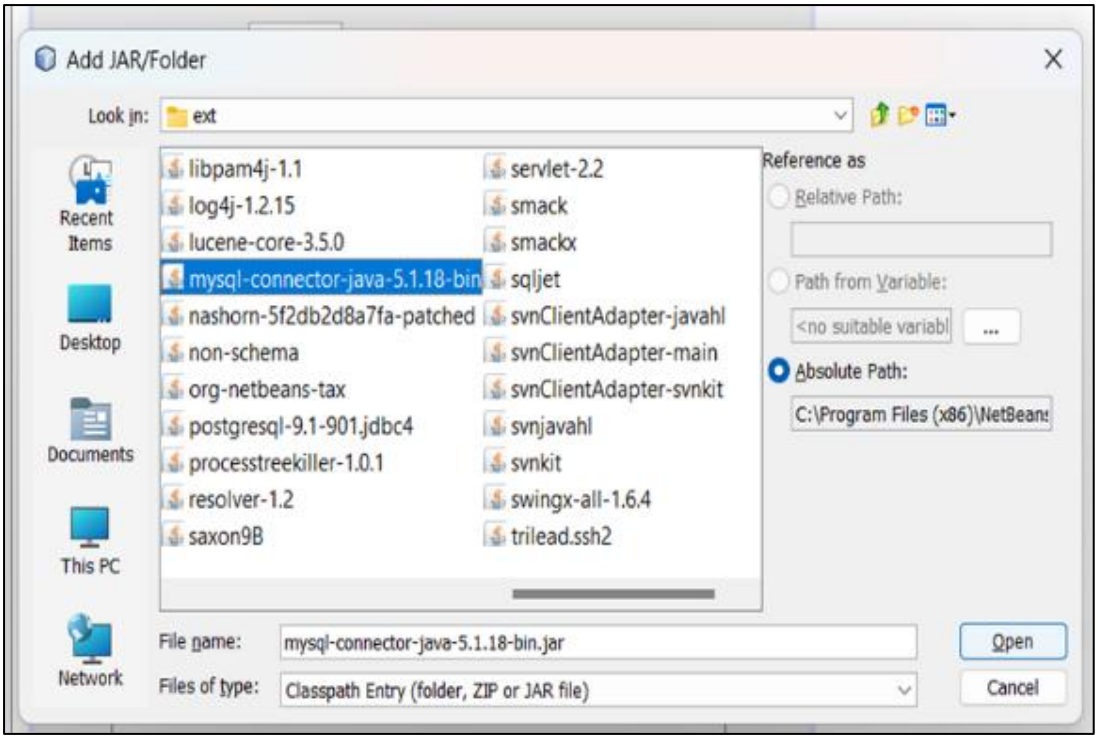
Establish a database connection within the Java NetBeans environment, the following steps are necessary: **DriverManager Class, Connection Class, and Statement Class.**

1. Tools-->Libraries, and add MySQL JDBC connector.
2. Copy the Library *ClassPath* and click OK.

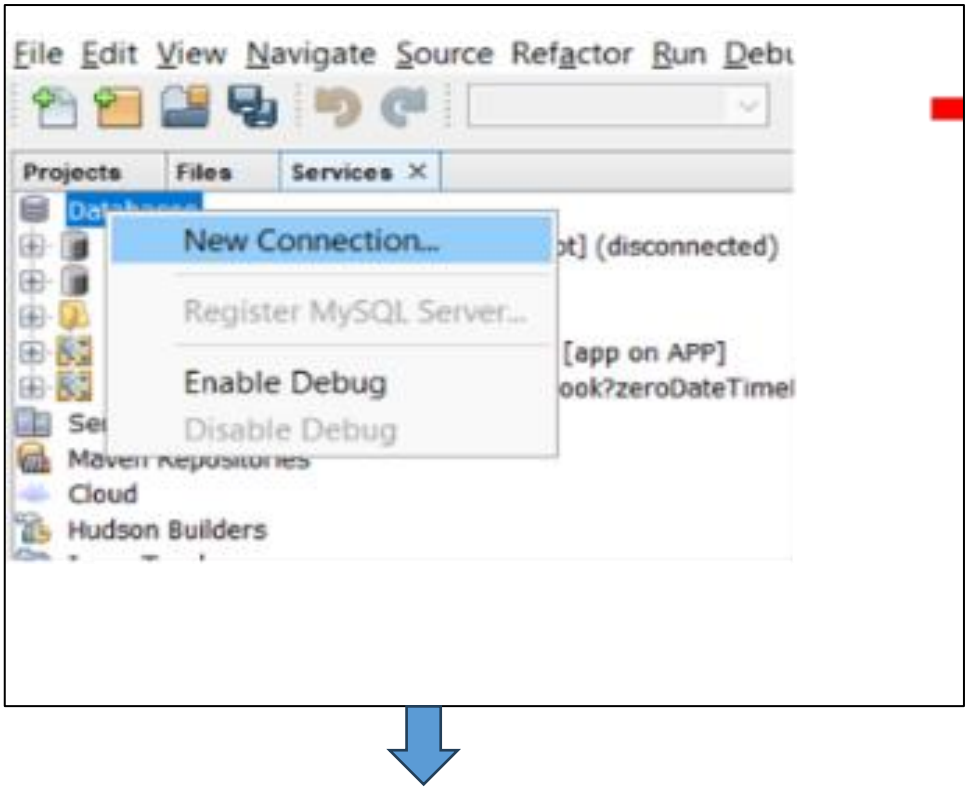


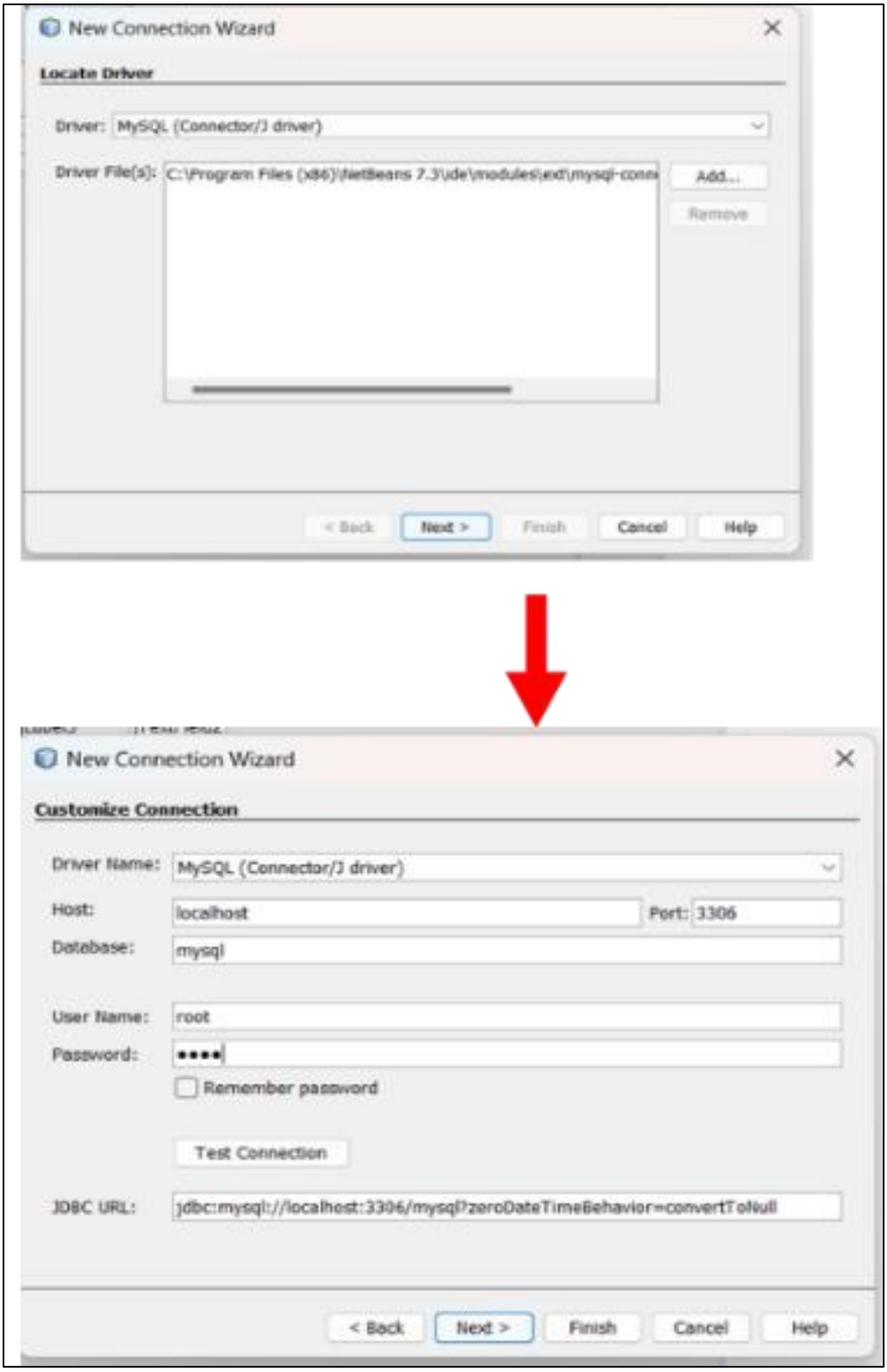
Projects --> Libraries. Right-click and choose *'Add Jar/Library.'* Then, navigate to and select the previously recorded Library *classpath*.



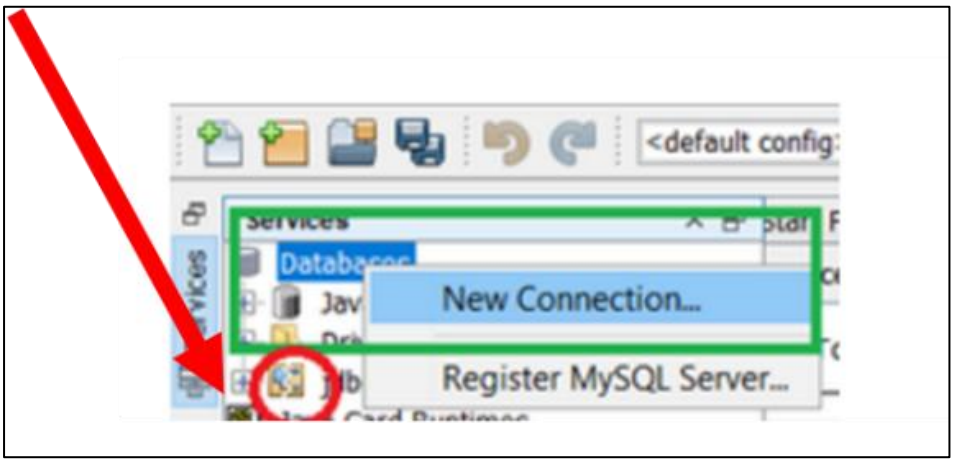


Windows-->Services-->Databases. Enter the required MySQL username and password credentials




New Connection Wizard
Locate Driver
Driver: MySQL (Connector/J driver)
Driver File(s): C:\Program Files (x86)\NetBeans 7.3\ide\modules\ext\mysql-conn
Add...
Remove
< Back
Next >
Finish
Cancel
Help
New Connection Wizard
Customize Connection
Driver Name: MySQL (Connector/J driver)
Host: localhost
Port: 3306
Database: mysql
User Name: root
Password: ••••
Remember password
Test Connection
JDBC URL: jdbc:mysql://localhost:3306/mysql?zeroDateTimeBehavior=convertToNull
< Back
Next >
Finish
Cancel
Help

If the *'Test Connection'* is successful, the connector logo will visually indicate a successful connection



Java code 1: Double-click on the jButton1 ('Insert') button. This action will direct you to the source code tab, then enter the following code:

```
import java.sql.*;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

public class SchoolRecord extends javax.swing.JFrame {

    Connection con = null;
    Statement stmt = null;
    ResultSet rs = null;
    // JButton1 Code starts from here
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/student", "root",
            "root");
        stmt = con.createStatement();
        String rollno = jTextField1.getText();
        String name = jTextField2.getText();
        String class = jTextField3.getText();
        String sec = jTextField4.getText();
        String adr = jTextArea1.getText();
        String INSERT = "INSERT INTO RECORD VALUES('"
                        + rollno + "','" + name + "','"
                        + class + "','" + sec + "','" + adr
                        + "');";
        stmt.executeUpdate(INSERT);
        JOptionPane.showMessageDialog(
            this, "Record Added Successfully");
        jButton1.setEnabled(true);
    }
    catch (Exception e) {
        JOptionPane.showMessageDialog(
            this, "Error In Connectivity");
    }
```

Considerations:

- ***Connection Class:*** Active connection between a Java program and database.
- ***Statement Class*:** Create and execute SQL commands and provides a set of rules for interacting with the database using SQL.
- *ResultSet:* Holds the data retrieved from the database, organized in a table format.
- ***JDBC Driver Registration**:* For MySQL databases, use *the forName("com.mysql.jdbc.Driver")* method to register the MySQL JDBC driver.
- ***getConnection:*** Create a link between the Java program and the database. The database name, username, and password should be provided to authenticate and gain access, resulting in a *"Connection object*" that represents a successful connection to the database.
- ***Query Execution**: The executeQuery() method* is used to execute *SELECT* statements, returning a *ResultSet* object containing the retrieved data. The *executeUpdate() method* is used to execute *INSERT, UPDATE, DELETE*, and other SQL statements that modify the database.
- ***Data Extraction:*** To access individual data elements within the *ResultSet,* we utilize appropriate getter methods such as *.getString(), .getInt(),* etc., based on the data type of the desired element.
- ***while(rs.next()):*** The *next() method* of the *ResultSet* object advances the cursor to the subsequent row, enabling iterative access to each row of data within the result set.
- ***Closing open databases:*** Closing the *ResultSet (rs.close()),* the Statement *(stmt.close()), and the Connection (con.close())* objects, which release the associated resources and prevent resource leaks.

Code to display all data within *the jButton2 ActionPerformed* event handler. This is triggered by double-clicking the *"View Data"* button in the design area.

```
DefaultTableModel model
    = (DefaultTableModel)jTable1.getModel();
try {
    Class.forName("com.mysql.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student", "root",
        "root");
    String query = "SELECT* FROM RECORD;";
    ResultSet rs = stmt.executeQuery(query);
    while (rs.next()) {
        String rollno = rs.getString("rollno");
        String name = rs.getString("name");
        String class = rs.getString("class");
        String sec = rs.getString("section");
        String adr = rs.getString("address");
        model.addRow(
            new Object[] { rollno, name, class, sec, adr });
    }
    rs.close();
    stmt.close();
    con.close();
}
catch (Exception e) {
    JOptionPane.showMessageDialog(this,
                                  "Error In Connectivity");
}
```

Code to clear all text fields, text areas, and table content within the *jButton3 ActionPerformed* event handler. This is triggered by double-clicking the *"Clear" button* in the design area.

```
jTextField1.setText("");
jTextField2.setText("");
jTextField3.setText("");
jTextField4.setText("");
jTextArea1.setText("");
DefaultTableModel dm
    = (DefaultTableModel)jTable1.getModel();
dm.getDataVector().removeAllElements();
jTable1.repaint();
```

Code for exit functionality within the event handler for the *"jButton4"* button. This is automatically invoked when the *"Exit"* button in the design area is double-clicked.

```
System.exit(0);
```



APPLICATION

Fill in the blanks with the correct term.

1. _____ is a collection of large amounts of related data organized in tables.
2. JDBC stands for _____.
3. The _____ component in JDBC acts as a communication channel between a Java program and a database.
4. The getConnection() method in JDBC is used to _____ to a database.
5. To iterate through multiple rows of data in a JDBC result set, you can use a loop with the _____ method.
6. Closing resources like the result set, statement, and connection in JDBC is crucial for _____.
7. JDBC drivers enable _____ applications to connect to databases.

Excellent work students ! You have successfully completed and mastered all the lessons in this module. Keep up the great work!

MODULE ASSESSMENT

1. Write a program that will compute the area of a circle using the formula: Area = π × r2 . Note that user input will come from the console (system), while the output will be written to text file “area.txt” At the end of the program, prompts the user that the area was written into the text file.

2. Design and Implement a Basic System with CRUD Operations.

MODULE SUMMARY

Java's file handling capabilities are essential for working with external data. It allows programs to read from and write to files, enabling data persistence beyond program execution. Streams, which are sequences of data, are the foundation for file I/O operations.

There are two main stream types: InputStream for reading data and OutputStream for writing data. Java supports fundamental file operations like creating new files, reading existing file contents, writing data to files, and deleting files. By mastering these techniques, you can empower your Java programs to effectively interact with external data storage.

JDBC is a powerful tool in Java applications for interacting with databases. It acts as a bridge between Java programs and various tabular data sources. Through JDBC APIs, developers can leverage SQL statements to perform CRUD (Create, Read, Update, Delete) operations on databases. Key components like JDBC drivers, Connection objects, Statements, and ResultSets facilitate communication and data manipulation.

JDBC offers advantages like enabling enterprise-level data management without additional software or client-side configuration. Overall, JDBC empowers Java applications to effectively interact with and manage data stored in relational databases.

REFERENCES

Burd, B. (2021). *Beginning programming with java for dummies*. John Wiley & Sons, Inc.

Budd, T.A. (2021). *An Introduction to object-oriented programming*. Addison Wesley.

Sarcar, V. (2020). *Interactive object-oriented programming in java*. Springer Science.

3G E-Learning. (2023). *Object-oriented Programming*.